

UNIT III

MEMORY MANAGEMENT

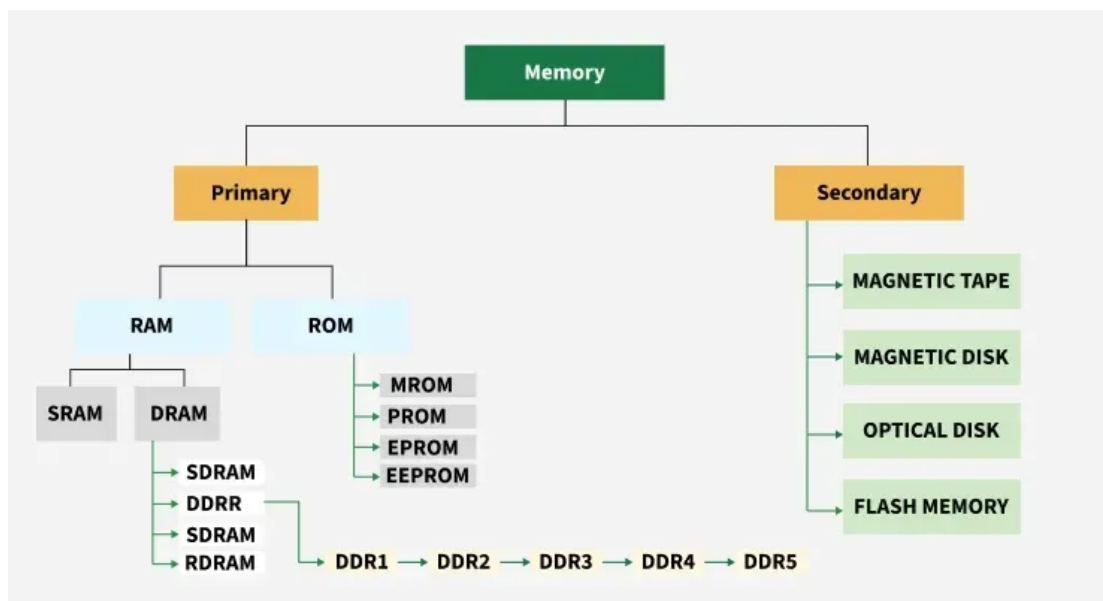
Main Memory - Swapping - Contiguous Memory Allocation – Paging - Structure of the Page Table - Segmentation, Segmentation with paging; Virtual Memory - Demand Paging – Copy on Write – Page Replacement - Allocation of Frames – Thrashing.

Memory is an essential component of a computer system, responsible for storing data and instructions needed for processing. It enables the CPU to execute programs efficiently and ensures smooth system operation.

- **Memory Cell:** Smallest unit storing 1 bit of data with a unique address.
- **Word & Byte:** A word is a group of bits; 1 byte = 8 bits.
- **Capacity:** Total number of bits a memory can hold.

Memory is classified into primary and secondary types based on speed, accessibility, and volatility.

- **Primary Memory:** Directly accessible by the CPU; fast but limited in capacity.
- **Secondary Memory:** Used for long-term storage; slower but larger in capacity.



RAM (Random Access Memory): RAM is the computer's main memory used for temporary storage of active programs and data. Data is lost when power is off. It provides fast CPU access, improving multitasking and performance.

Types of RAM

- SRAM (Static RAM): Fast, used in cache, retains data while powered.
- DRAM (Dynamic RAM): Slower, needs periodic refresh; main memory. Includes SDRAM and DDR series.

ROM (Read-Only Memory): ROM is non-volatile memory that stores essential instructions permanently. It holds system firmware and boot instructions.

Types of ROM

- MROM: Pre-programmed at manufacture
- PROM: User-programmable once
- EPROM: UV-erasable
- EEPROM: Electrically erasable
- Flash Memory: Fast, used in SSDs and USB drives

Secondary Memory: Secondary memory provides long-term data storage and is not directly accessed by the CPU.

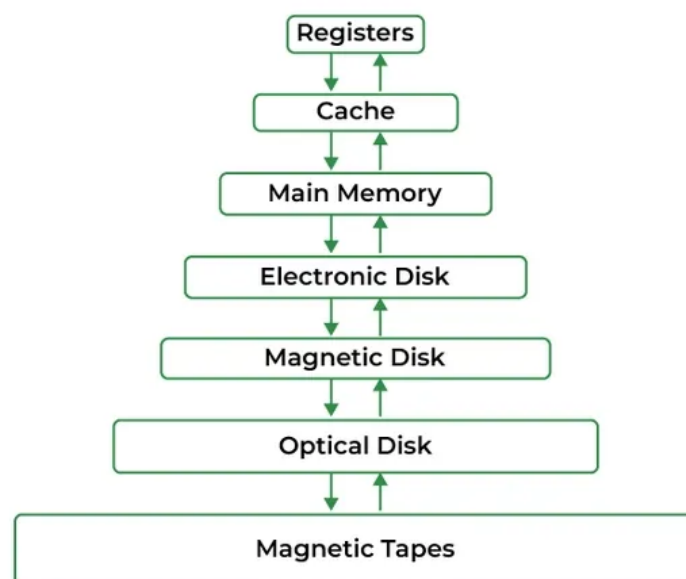
- Examples: HDD, SSD, optical discs (CD/DVD/Blu-ray), USB drives, flash cards, magnetic tapes, cloud storage.
- Characteristics: Non-volatile, slower, high capacity, used for storing operating systems, software, and user files.

Differences Among RAM, ROM, and Secondary Memory

RAM	ROM	Secondary Memory
Volatile	Non-volatile	Non-volatile
Temporary workspace	Permanent instructions	Long-term storage

RAM	ROM	Secondary Memory
Fast	Moderate	Slow
Read/Write	Mostly Read-only	Read/Write
DRAM, SRAM	PROM, EPROM, EEPROM	HDD, SSD, USB

Memory management in operating systems is the core function of controlling, organizing, and coordinating computer primary memory (RAM) to efficiently allocate, track, and protect memory resources for running processes. It manages the movement of data between main memory and disk, ensuring maximum system performance, minimizing fragmentation, and providing security through process isolation.



Memory management in simpler terms is the process of controlling and organising a computer's memory by allocating portions, called blocks, to different executing programmes to improve the overall system performance.

- The most important function of an operating system is to manage primary memory.
- It supports multiple processes simultaneously in memory.

- Protects processes from unauthorised access.
- Enables swapping and virtual memory efficiently.

Role of Main Memory in OS:

The main memory is central to the operation of a Modern Computer. Main Memory is a large array of words or bytes, ranging in size from hundreds of thousands to billions. Main memory is a repository of rapidly available information shared by the CPU and I/O devices. Main memory is the place where programs and information are kept when the processor is effectively utilizing them. Main memory is associated with the processor, so moving instructions and information into and out of the processor is extremely fast. Main memory is also known as RAM (Random Access Memory). This memory is volatile. RAM loses its data when a power interruption occurs.

- Main memory (RAM) holds instructions and data of processes that are currently executing; the CPU can only execute instructions that are in main memory.
- The OS decides which processes are in memory, where they are placed, and when they are moved in or out (swapped) to/from secondary storage.
- It must keep track of which parts of memory are allocated or free, and ensure that one process cannot corrupt another or the OS itself.

Why Memory Management is Required?

- Allocate and de-allocate memory before and after process execution.
- To keep track of used memory space by processes.
- To minimize fragmentation issues.
- To proper utilization of main memory.
- To maintain data integrity while executing of process.

Address Spaces and Address Binding

Every process uses a logical (virtual) address space – addresses generated by the CPU as the program runs. Physical addresses refer to actual locations in RAM; the OS and hardware must map logical addresses to physical ones.

Logical and Physical Address Space

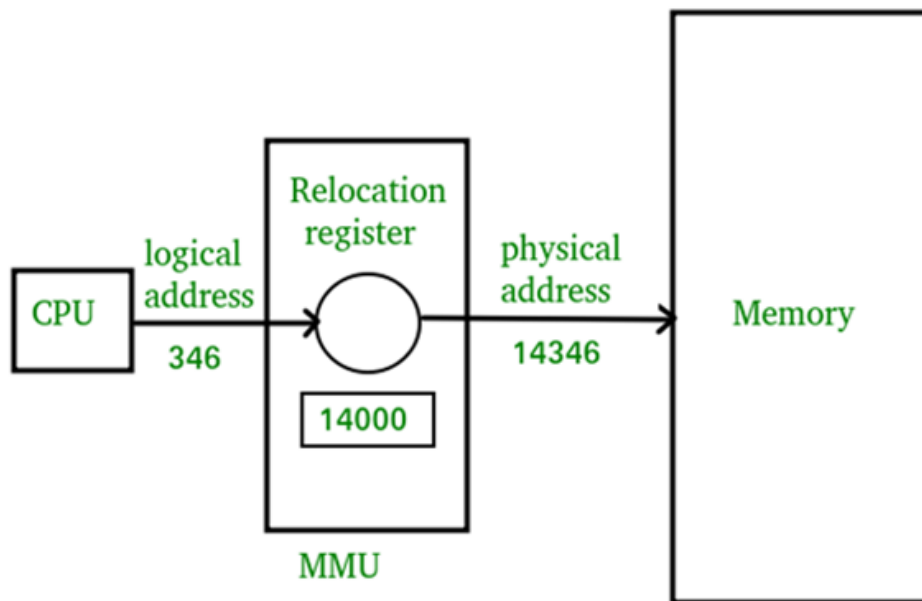
Logical Address Space: An address generated by the CPU is known as a “Logical Address”. It is also known as a Virtual address. Logical address space can be defined as the size of the process. A logical address can be changed.

Physical Address Space: An address seen by the memory unit (i.e. the one loaded into the memory address register of the memory) is commonly known as a “Physical Address”. A Physical address is also known as a Real address. The set of all physical addresses corresponding to these logical addresses is known as Physical address space. A physical address is computed by MMU. The run-time mapping from virtual to physical addresses is done by a hardware device Memory Management Unit(MMU). The physical address always remains constant.

The **Memory Management Unit (MMU)** is a critical hardware component, usually integrated into the CPU, that manages data flow between the processor and memory. It enables virtual memory by translating CPU-generated virtual addresses into physical RAM addresses. Key functions include memory protection, paging, and improving system performance.

Key Aspects of MMU in Operating Systems:

- **Address Translation:** The core function is converting logical (virtual) addresses to physical addresses in real-time.
- **Memory Protection:** Ensures that processes cannot access or interfere with the memory space of other applications, enhancing system stability
- **Virtual Memory Support:** Enables running programs larger than the physical memory by mapping virtual pages to physical RAM or disk storage.
- **Translation Lookaside Buffer (TLB):** A fast, small hardware cache within the MMU that stores recent virtual-to-physical translations to speed up the process.
- **Page Fault Handling:** If a required page is not in physical memory, the MMU signals the operating system via an interrupt.



Address binding is the process of mapping the addresses used in a program (logical/virtual addresses) to actual locations in main memory (physical addresses) so the CPU can access instructions and data correctly.

- When you write and compile a program, you use symbolic names and logical addresses, not real RAM locations.
- The operating system and related tools (compiler, linker, loader, MMU hardware) bind these logical addresses to physical addresses at some stage, so every logical address the CPU generates has a unique physical address in RAM.

Types/stages of address binding

1. Compile-time binding

- Done by the compiler when the final load address is known in advance.
- The compiler generates absolute addresses; the program must always be loaded at that fixed location, or it cannot run.

2. Load-time binding

- Used when the final load address is not known at compile time.
- The compiler generates relocatable addresses; when the loader places the program in memory, it adds a base value to all relocatable addresses to get the final physical addresses.

3. Execution-time binding

- Used when a process may move in memory while it is running (as in modern multiprogramming with relocation, paging, segmentation).
- A Memory-Management Unit (MMU) performs address translation on the fly, adding a relocation/base value or using page/segment tables to convert each logical address to a physical address during execution.

Execution-time binding is what enables features like relocation, swapping, and virtual memory, because the physical location can change without modifying the program itself.

Address binding vs loading/linking

- Address binding: Mapping of logical/virtual addresses used in a program to physical addresses in main memory, done at compile time, load time, or execution time using the MMU and OS.
- Loading/linking: Concerned with *when* a program's code (and libraries) are placed into memory and *when/how* separate object files or libraries are combined into a single executable image.

Static and Dynamic Loading

Loading a process into the main memory is done by a loader. There are two different types of loading :

Static Loading: Static Loading is basically loading the entire program into a fixed address. It requires more memory space.

Dynamic Loading: The entire program and all data of a process must be in physical memory for the process to execute. So, the size of a process is limited to the size of physical memory. To gain proper memory utilization, dynamic loading is used. In dynamic loading, a routine is not loaded until it is called. All routines are residing on disk in a relocatable load format. One of the advantages of dynamic loading is that the unused routine is never loaded. This loading is useful when a large amount of code is needed to handle it efficiently.

Static and Dynamic Linking

To perform a linking task a linker is used. A linker is a program that takes one or more object files generated by a compiler and combines them into a single executable file.

Static Linking: In static linking, the linker combines all necessary program modules into a single executable program. So there is no runtime dependency. Some operating systems support only static linking, in which system language libraries are treated like any other object module.

Dynamic Linking: The basic concept of dynamic linking is similar to dynamic loading. In dynamic linking, “Stub” is included for each appropriate library routine reference.

A stub is a small piece of code. When the stub is executed, it checks whether the needed routine is already in memory or not. If not available then the program loads the routine into memory.

Overlays:

- Enable a process larger than the amount of memory allocated to it.
- At a given time, the needed instructions & data are to be kept within a memory.

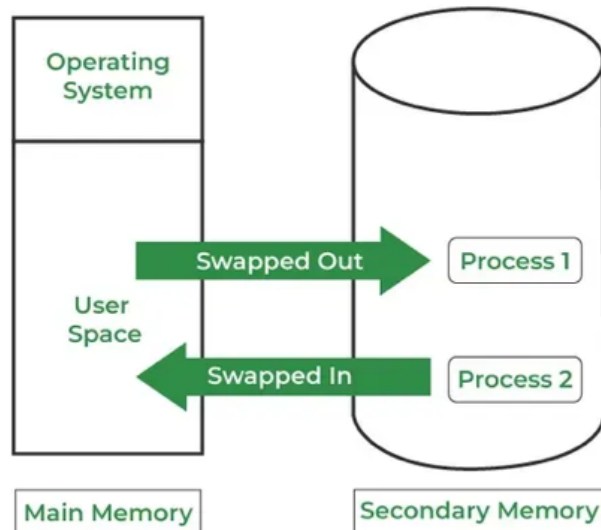
Swapping

Swapping is a memory management technique in which a process is temporarily moved from main memory (RAM) to secondary storage (disk) and vice versa. This allows the operating system to manage limited RAM effectively and run multiple processes concurrently in a multiprogramming environment.

Advantages:

- Reduces memory fragmentation by moving processes in and out of RAM.
- Allows large processes to run even if they exceed available RAM.
- Frees RAM temporarily by suspending inactive or waiting processes.
- Supports virtual memory to give processes more memory than physically available.

Example: RAM has 4 GB with Process A and Process B running, each using 2 GB. When Process C, which also requires 2 GB, needs to run, there is no free memory. The operating system swaps Process A out to the disk and loads Process C into RAM. After Process C finishes, Process A is brought back into memory to continue execution.



How swapping works:

1. A process is moved into RAM from disk when it is ready to execute.
2. After the process has run for a while (or if higher-priority processes need memory), it may be temporarily swapped out to disk.
3. The CPU scheduler decides which process to swap in and out, often based on priority or scheduling policies.
4. When the swapped-out process is needed again, it is swapped back into memory to continue execution.

A process can be swapped temporarily out of memory to a backing store (SWAP OUT) and then brought back into memory for continued execution (SWAP IN).

- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users & it must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- **Transfer time** :
 - Major part of swap time is transfer time
 - Total transfer time is directly proportional to the amount of memory swapped.

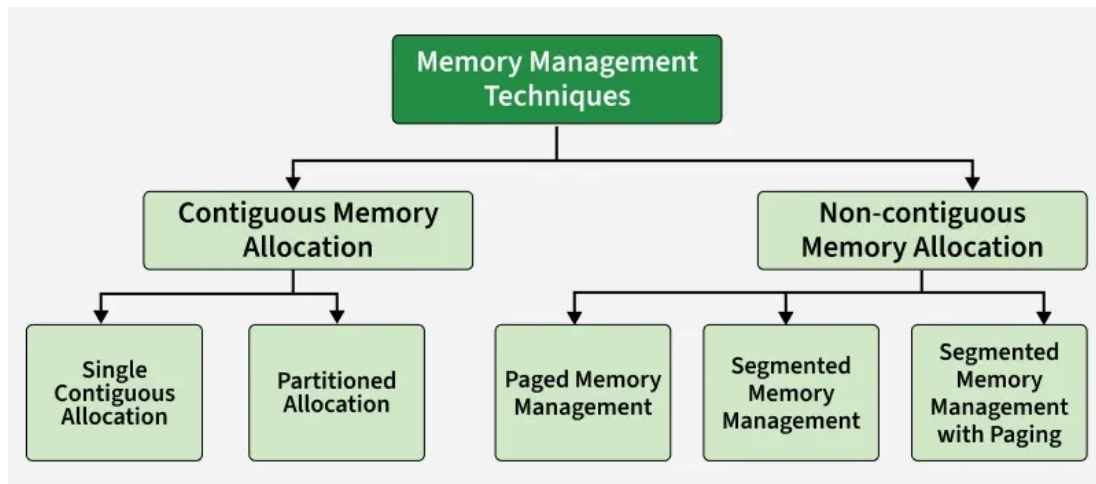
- **Example:** Let us assume the user process is of size 1MB & the backing store is a standard hard disk with a transfer rate of 5MBPS.

Transfer time = $1000\text{KB} / 5000\text{KB per second} = 1/5 \text{ sec} = 200\text{ms}$

Advantages	Disadvantages
Reduces waiting time by using swap space as an extension of RAM, keeping the CPU busy.	Risk of data loss if the system crashes during swapping.
Frees main memory by moving inactive processes to secondary storage.	Increased page faults due to frequent swapping reduce performance.
Allows multiple processes to run using a single main memory with a swap partition.	Disk access is slower than RAM, causing execution delays.
Enables large programs to run on systems with limited physical memory.	Excessive swapping (thrashing) causes severe performance degradation.
Prevents memory overload by ensuring active processes have enough RAM.	CPU may spend more time swapping than executing processes.

Techniques in memory allocation are strategies the operating system uses to decide how to give RAM to processes, **where** to place them, and **when** to move or **resize** their memory so that overall utilization remains high and fragmentation low. They can be broadly categorized into:

1. Contiguous Allocation Techniques
2. Non-Contiguous Allocation: Paging
3. Non-Contiguous Allocation: Segmentation (and Segmentation with Paging)
4. Dynamic Allocation and Relocation at Runtime
5. Virtual Memory and Frame Allocation
6. Kernel Memory Allocation



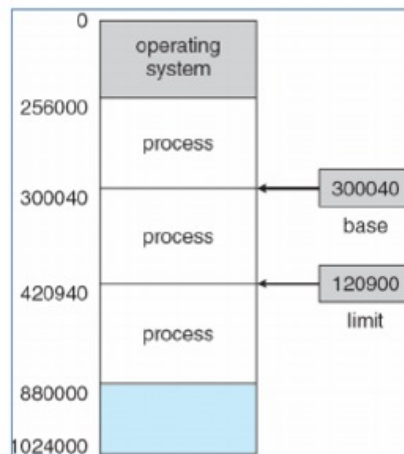
Contiguous Memory Allocation

Contiguous memory allocation is a memory management method where each process is given a single, continuous block of memory. This means all the data for a process is stored in adjacent memory locations.

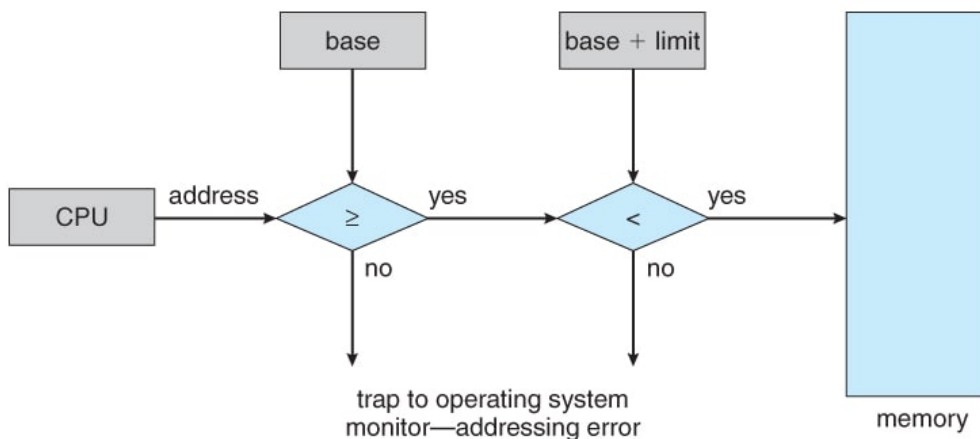
- In contiguous allocation, each process occupies one continuous block of physical memory, so the OS must protect:
 - a) The operating system region from user processes.
 - b) Each user process from accessing another process's memory.
- Protection is provided using a relocation (base) register and limit register:
 - The relocation/base register holds the smallest physical address of the process (start of its allocated block).
 - The limit register holds the size/range of the process's logical address space.
- For every logical address generated by the CPU:
 - The hardware checks that the logical address is less than the limit (within the process's range).
 - If valid, it adds the base to the logical address to form the physical address; otherwise it raises a trap (protection fault).

What is the Base register and what is the Limit register?

- **Base register:** Specifies the smallest legal physical memory address.
- **Limit register:** Specifies the size of the range.
- A pair of base and limit registers specifies the logical address space.
- The base and limit registers can be loaded only by the **operating system**.
- Ex: If the base register holds 300040 and the limit register is 120900, then the program can legally access all addresses from 300040 through 420939 (inclusive).



Hardware address protection uses **base** and **limit** registers to define a process's legal physical memory space, ensuring it cannot access or corrupt memory belonging to other processes or the operating system.



Partition Allocation Methods: To gain proper memory utilization, memory allocation must be allocated efficient manner. One of the simplest methods for allocating memory is to divide memory into several fixed-sized partitions and each partition contains exactly one process. Thus, the degree of multiprogramming is obtained by the number of partitions.

Fixed partition allocation: Memory is divided into fixed-sized partitions during system initialization. Each partition can hold only one process. Fixed partition memory allocation, or static partitioning, divides main memory into non-overlapping, fixed-size, or variable-sized regions at system boot, restricting the number of concurrent processes to the number of partitions. It simplifies management by placing processes directly into available, adequately sized partitions. While easy to implement, it causes significant internal fragmentation and limits multitasking capacity. In this allocation once partitions are defined operating system keeps track of the status of memory partitions it is done through a data structure called a partition table.

Starting Address of Partition	Size of Partition	Status
0k	200k	allocated
200k	100k	free
300k	150k	free
450k	250k	allocated

Advantages

- **Simplicity:** Easy to implement and requires minimal OS overhead to manage.
- **Low Overhead:** No complex algorithms are needed for allocation.
- **Predictable:** Provides a consistent, predictable amount of memory for each process.

Disadvantages

- **Internal Fragmentation:** If a process is smaller than the assigned partition, the remaining space in that partition is wasted.
- **Limited Multiprogramming:** The maximum number of active processes is strictly limited by the number of partitions defined at system start.
- **Partition Size Constraint:** Processes cannot be loaded if they are larger than the largest available partition.

Dynamic Partition Allocation: Dynamic partition memory allocation (or variable partitioning) is a contiguous memory management technique where the operating system allocates exactly the amount of memory requested by a process at runtime. Unlike fixed partitioning, this method eliminates internal fragmentation because partitions are created to fit the process size. However, it leads to external fragmentation, requiring algorithms like First-fit or Best-fit to manage free memory holes.

Hole – block of available memory; holes of various size are scattered throughout memory.

Internal fragmentation is the wastage of memory that occurs when fixed-sized memory blocks are allocated to processes, but the process does not use the entire allocated block. The unused portion inside the allocated block remains idle, resulting in poor memory utilization, while **external fragmentation** is a problem in memory management where **free memory is divided into small, non-contiguous blocks**. Even though there may be enough total free memory to run a new program, the memory is scattered in tiny pieces, so it's impossible to find a single, large block for the program to use. This leads to wasted memory and poor system performance.

Solutions for Fragmentation:

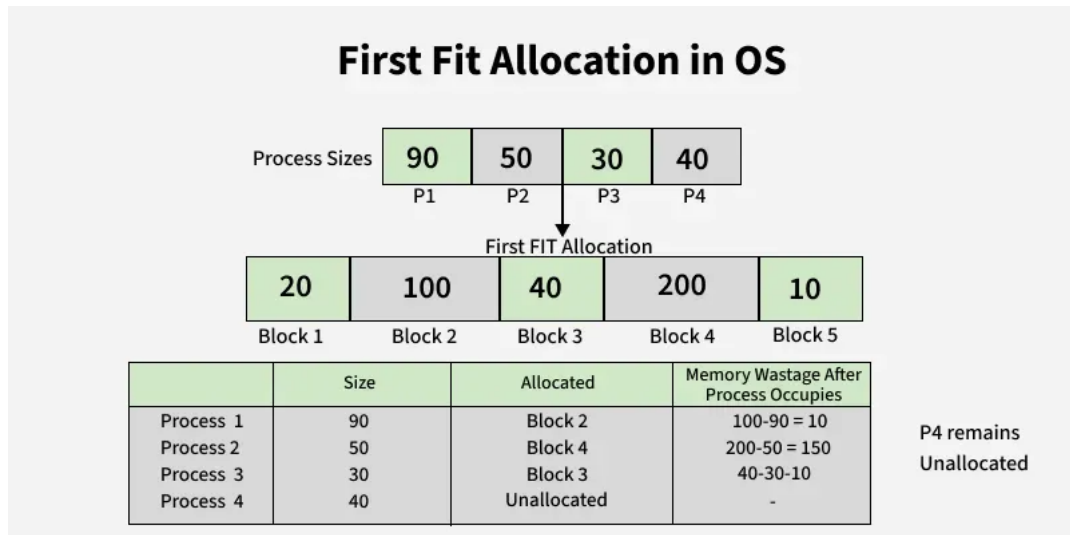
1. **Coalescing** : Merge the adjacent holes together.
2. **Compaction**: Move all processes towards one end of memory, hole towards other end of memory, producing one large hole of available memory. This scheme is expensive as it can be done if relocation is dynamic and done at execution time.
3. Permit the logical address space of a process to be **non-contiguous**. This is achieved through two memory management schemes namely **paging** and **segmentation**.

*In **Partition Allocation**, when there is more than one partition freely available to accommodate a process's request, a partition must be selected. To choose a particular partition, a partition allocation method is needed. A partition allocation method is considered better if it avoids internal fragmentation.*

How to satisfy a request of size $_n$ ' from a list of free holes?

There are different Placement Algorithm:

1. **First Fit:** First fit is a dynamic memory allocation algorithm that searches for the first available, sufficiently large memory partition to hold a process, scanning from the beginning of memory. It is fast, efficient in speed, and generally reduces the frequency of creating unusable, tiny gaps compared to other methods.



2. **Best Fit:** The Best Fit Algorithm allocates searches the entire memory to find the smallest available block that is large enough to accommodate the process. This means that, the algorithm looks for the block that will leave the least amount of unused space after allocation.

Job Number	Memory Requested
J1	20 K
J2	200 K
J3	500 K
J4	50 K

Memory location	Memory block size	Job number	Job size	Status	Internal fragmentation
10567	30 K	J1	20 K	Busy	10 K
30457	50 K	J4	50 K	Busy	None
300875	200 K	J2	200 K	Busy	None
809567	700 K	J3	500 K	Busy	200 K
Total available :	980 K	Total used :	770 K		210 K

3. **Worst Fit:** In **Worst-Fit Memory Allocation** technique, the process traverses the whole memory and always search for the largest hole/partition, and then the process is placed

in that hole/partition. It is a slow process because it has to traverse the entire memory to search the largest hole.

4. Next Fit: Next fit is a modified memory management algorithm that allocates processes to the first available, sufficiently large memory block, starting from the last allocated position rather than the beginning. It uses a moving (roving) pointer to reduce search time, improve load balancing, and minimize fragmentation near the start of memory.

Non-Contiguous Allocation: Paging

Paging is the process of moving parts of a program, called pages, from secondary storage into the main memory (RAM). The main idea behind paging is to break a program into smaller fixed-size blocks called pages.

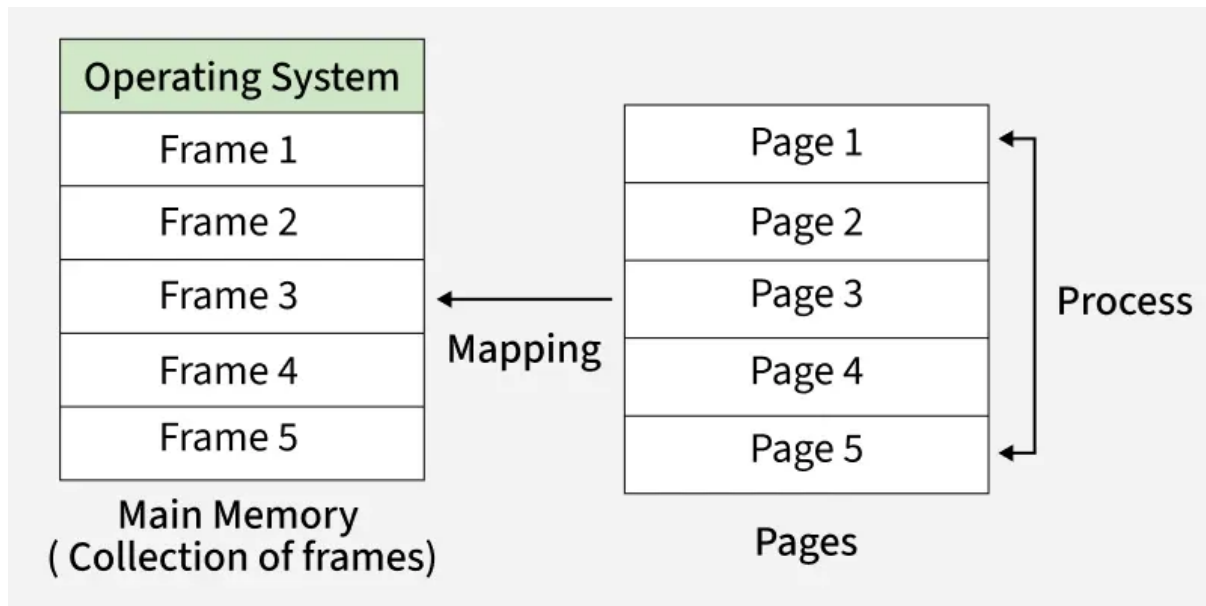
Paging is a memory management scheme that eliminates the need for contiguous allocation of physical memory by dividing processes into fixed-size blocks called **pages** and physical memory into corresponding **frames**. It enables non-contiguous storage, reducing external fragmentation while allowing efficient memory utilization through a page table mapping logical to physical addresses.

Advantages:

- Eliminates External Fragmentation: Since any page can go into any frame, there is no need for contiguous memory allocation.
- Efficient Memory Utilization: Pages are loaded into memory only when needed, supporting larger programs than available RAM (virtual memory).
- Simplified Allocation: The OS only needs to keep track of free frames.

Disadvantages:

- Internal Fragmentation: If a process does not need a full page, memory is wasted within the last frame.
- Page Table Overhead: Maintaining page tables for each process requires extra memory.
- Increased Access Time: Memory access takes longer due to the need to access the page table first.



To keep track of where each page is stored in memory, the operating system uses a page table.

Paging addresses common challenges in allocating and managing memory efficiently. Why paging is needed as a Memory Management technique:

- **Memory isn't always available in a single block:** Programs often need more memory than what is available in a single continuous block. Paging breaks memory into smaller, fixed-size pieces, making it easier to allocate scattered free spaces.
- **Processes size can increase or decrease:** programs don't need to occupy continuous memory, so they can grow dynamically without the need to be moved.

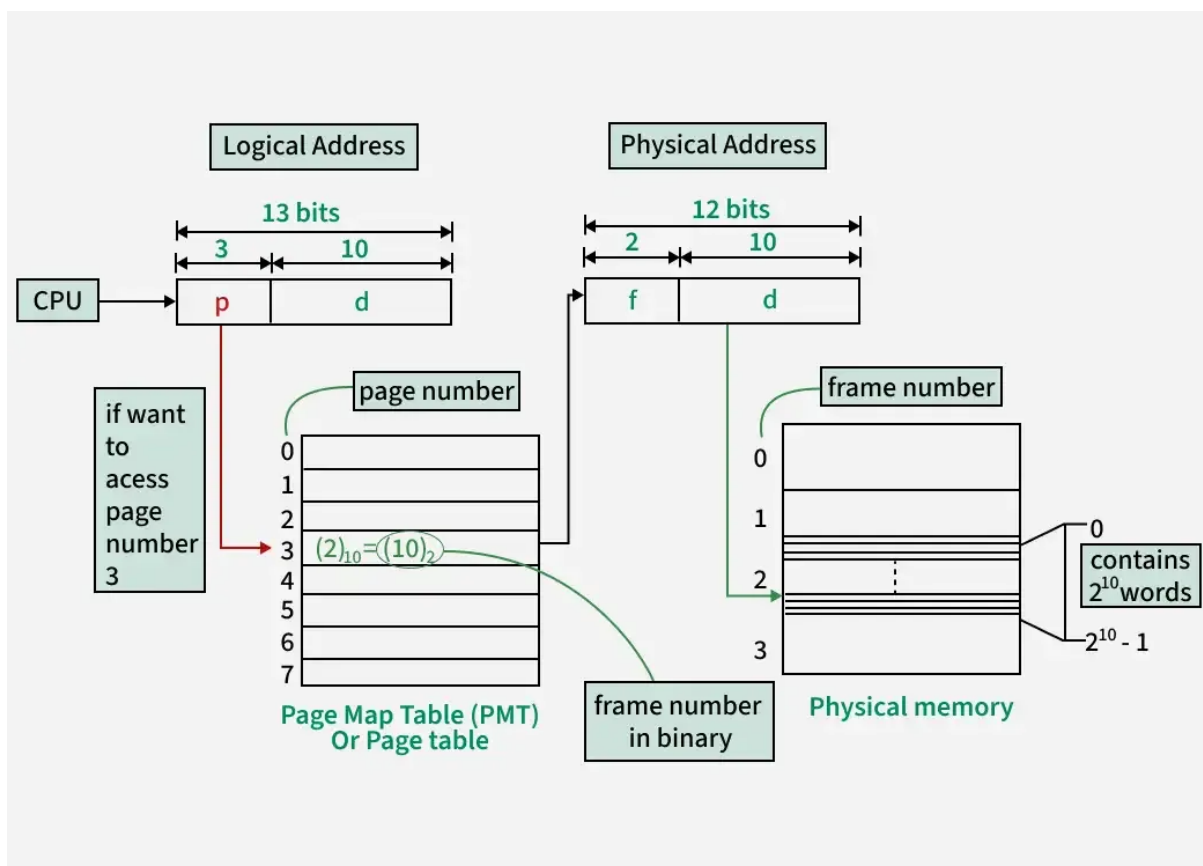
Important Features of Paging

- **Logical to physical address mapping:** Paging divides a process's logical address space into fixed-size pages. Each page maps to a frame in physical memory, enabling flexible memory management.
- **Fixed page and frame size:** Pages and frames have the same fixed size. This simplifies memory management and improves system performance.
- **Page table entries:** Each logical page is represented by a page table entry (PTE). A PTE stores the corresponding frame number and control bits.
- **Number of page table entries:** The page table has one entry per logical page. Thus, its size equals the number of pages in the process's address space.

- **Page table stored in main memory:** The page table is kept in main memory. This can add overhead when processes are swapped in or out.

How Paging works:

When a process requests memory, the operating system allocates one or more page frames to the process and maps the process's logical pages to the physical page frames. When a program runs, its pages are loaded into any available frames in the physical memory. Each program has a page table, which the operating system uses to keep track of where each page is stored in physical memory. When a program accesses data, the system uses this table to convert the program's address into a physical memory address.



Steps Involved in Paging :

- **Step 1 Divide Memory :** Logical $\rightarrow$ Pages, Physical $\rightarrow$ Frames .
- **Step 2 Allocate Pages :** Load pages into available frames.
- **Step 3 Page Table :** Map logical pages to physical frames.
- **Step 4 Translate Address :** Convert logical to physical address.

- **Step 5 Handle Page Fault** : Load missing pages from disk.
- **Step 6 Run Program** : CPU uses page table during execution.

*If Logical Address Space = 128 M words = 2^{27} * 220 words, then Logical Address = $\log_2 2^{27}$*
 = 27 *bits*

*If Physical Address Space = 16 M words = 2^{24} * 220 words, then Physical Address = $\log_2 2^{24}$*
 = 24 *bits*

The mapping from virtual to physical address is done by the Memory Management Unit (MMU) which is a hardware device and this mapping is known as the paging technique.

- The Physical Address Space is conceptually divided into a number of fixed-size blocks, called frames.
- The Logical Address Space is also split into fixed-size blocks, called pages.
- Page Size = Frame Size

Example

- **Physical Address** = 12 bits, then Physical Address Space = 4 K words
- **Logical Address** = 13 bits, then Logical Address Space = 8 K words
- **Page size** = frame size = 1 K words (assumption)

Number of frames = Physical Address Space / Frame Size = $4K / 1K = 4 = 2^2$

Number of Pages = Logical Address Space / Page Size = $8K / 1K = 2^3$

The address generated by the CPU is divided into:

1. **Page number(p):** Number of bits required to represent the pages in Logical Address Space or Page number
2. **Page offset(d):** Number of bits required to represent a particular word in a page or page size of Logical Address Space or word number of a page or page offset.

A Physical Address is divided into two main parts:

1. **Frame Number(f):** Number of bits required to represent the frame of Physical Address Space or Frame number frame

2. **Frame Offset(d):** Number of bits required to represent a particular word in a frame or frame size of Physical Address Space or word number of a frame or frame offset.

Physical Address = (Frame Number << Number of Bits in Frame Offset) + Frame Offset
where "<<" represents a bitwise left shift operation.

Paging example for a 32-byte memory with 4-byte pages

Page size = 4 bytes

Physical memory size = 32 bytes i.e (4 X 8 = 32 so, 8 pages)

Logical address $_0'$ maps to physical address 20 i.e ((5 X 4) +0)

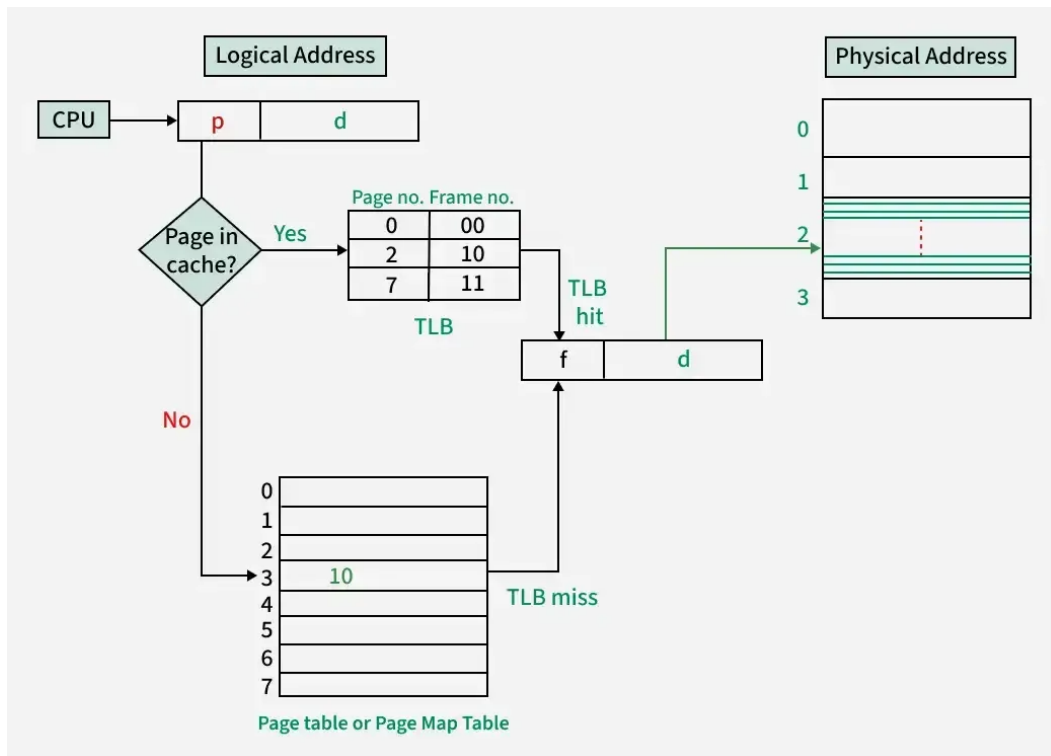
Where Frame no = 5, Page size = 4, Offset = 0

Allocation

- When a process arrives into the system, its size (expressed in pages) is examined.
- Each page of process needs one frame. Thus if the process requires $_n'$ pages, at least $_n'$ frames must be available in memory.
- If $_n'$ frames are available, they are allocated to this arriving process.
- The 1st page of the process is loaded into one of the allocated frames & the frame number is put into the page table.
- Repeat the above step for the next pages & so on.

Hardware implementation of Paging

The hardware implementation of the page table can be done by using dedicated registers. But the usage of the register for the page table is satisfactory only if the page table is small. If the page table contains a large number of entries then we can use **TLB**(translation Look-aside buffer), a special, small, fast look-up hardware cache. The simplest case is **Page-table base register (PTBR)**, is an index to point the page table.



- The TLB is associative, high-speed memory.
- Each entry in TLB consists of two parts: a tag and a value.
- When this memory is used, then an item is compared with all tags simultaneously. If the item is found, then the corresponding value is returned.
- TLB is more expensive.

Main memory access time = m , If page table are kept in main memory, **Effective access time** = $m(\text{for page table}) + m(\text{for particular page in page table})$

How Does the TLB Work?

When the CPU generates a virtual address, it first looks up the page number in the TLB.

- **TLB Hit:** It is the situation when, the page number is found in the TLB, the corresponding frame number is retrieved instantly.
- **TLB Miss:** It is the situation when, the page number is not found in the TLB, the CPU accesses the page table in main memory.

Step-by-Step Process

Steps in TLB Hit:

1. CPU generates a virtual (logical) address.
2. It is checked in TLB (present).
3. The corresponding frame number is retrieved, which now tells where the main memory page lies.

Steps in TLB miss:

1. CPU generates a virtual (logical) address.
2. It is checked in TLB (not present).
3. Now the page number is matched to the page table residing in the main memory (assuming the page table contains all PTE).
4. The corresponding frame number is retrieved, which now tells where the main memory page lies.
5. The TLB is updated with new PTE (if space is not there, one of the replacement techniques comes into the picture i.e either FIFO, LRU or MFU etc).

Effective memory access time(EMAT)

TLB is used to reduce adequate memory access time as it is a high-speed associative cache.

$$EMAT = h \times (c + m) + (1 - h) \times (c + n \times m)$$

where:

- h is the hit ratio of the TLB,
- m is the memory access time,
- c is the TLB access time and
- n represents the system level.

***Note:** A higher hit ratio hhh significantly reduces EMAT, making the system efficient.*

Benefits of Using TLB in Paging

- **Faster Address Translation:** Reduces the need for repeated page table accesses.
- **Better Performance:** Reduces average memory access time dramatically.

- **Efficient Memory Utilization:** Only the most frequently used PTEs are cached in the small, fast TLB.
- **Scalability:** Especially useful for large address spaces in 64-bit architectures.

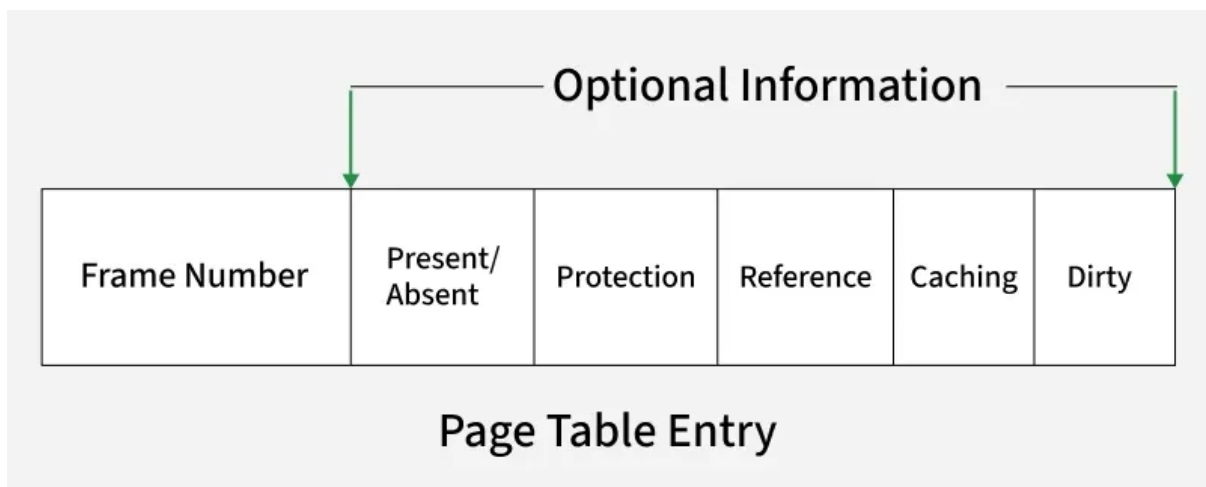
Protection:

Memory protection implemented by associating protection bit with each frame

- **Valid-invalid** bit attached to each entry in the page table:
 - “**valid (v)**” indicates that the associated page is in the process’ logical address space, and is thus a legal page
 - “**invalid (i)**” indicates that the page is not in the process’ logical address space

Structure of Page Table

A Page Table is a data structure that maps virtual addresses (used by processes) to physical addresses (actual locations in memory). The size and format of a PTE vary by system architecture and OS, but it always contains enough information for efficient memory management and protection.



Note: Each mapping in the table is stored in a Page Table Entry (PTE), which contains critical information required for efficient memory management, protection and address translation.

Common structures of a Page Table include:

- Hierarchical (multi-level) paging for large address spaces,
- Hashed page tables for 64-bit systems, and
- Inverted page tables to save space by mapping physical frames

A Page Table Entry (PTE) stores the necessary metadata about a page, allowing the system to manage memory safely and efficiently. The exact format and size of a PTE depend on the system's architecture (32-bit, 64-bit) and the operating system implementation, but typical fields include the following:

1. Frame Number

- Identifies the frame in physical memory where the page is stored.
- Number of bits required $= \log_2 \left(\frac{\text{Size of Physical Memory}}{\text{Frame Size}} \right)$

2. Present/Absent Bit (Valid/Invalid Bit)

Indicates whether the page is currently loaded in memory or not.

- **Present (1):** Page is in physical memory.
- **Absent (0):** Page fault occurs when accessed and the OS must load it from disk.

3. Protection Bits

- Define the allowed operations on the page:

Read (R)

Write (W)

Execute (X)

- These bits prevent unauthorized memory access
- Control access permissions (read, write, execute).
- Helps protect memory from unauthorized access.

4. Referenced Bit

- Set automatically by hardware when the page is accessed (read or written).
- Used by page replacement algorithms (e.g., LRU – Least Recently Used).

5. Caching Enabled/Disabled

- Determines whether the page should be cached.
- **Use case example:** Disable caching for memory-mapped I/O (keyboard or hardware registers) to always get fresh data.

6. Modified Bit (Dirty Bit)

- Indicates if the page has been written to.
- If set (1), the page must be written back to disk when evicted; otherwise, it can be discarded.

Example: For a 32-bit virtual address space with 4KB pages:

$$\text{Number of Pages} = \frac{\text{Total Words}}{\text{Words per Page}}$$

Here for words per page we have 4KB i.e $4KB = 2^{10} \times 2^2 = 2^{10+2} = 2^{12}$

12 bits words per page.

- Number of pages $= \frac{2^{32}}{2^{12}} = 2^{20}$ (about 1 million entries)
- If each PTE takes 4 bytes, total Page Table size $\approx 2^{20} \times 4 \text{ bytes} = 4 \text{ MB}$

Note: For 64-bit systems, a flat page table becomes too large, so hierarchical structures (multi-level page tables) are used to manage memory efficiently.

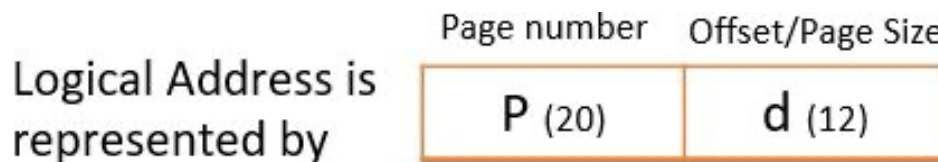
Hierarchical paging (or multilevel paging) in an OS is a memory management technique that divides large page tables into multiple levels (like two-level or three-level) to efficiently map vast logical address spaces, overcoming the issue of a single, huge page table that doesn't fit in memory by paging the page table itself. It breaks down a virtual address into parts (e.g., outer index, inner index, offset) to traverse these tables, reducing memory overhead and allowing systems to manage large virtual memories effectively.

1. **Address Breakdown:** A logical address is split into multiple parts. For a two-level system, it's often an outer page table index and an inner page table index (or offset within the inner table) plus the final offset within the data page.
2. **Outer Page Table:** The first level table (outer) points to the second-level page tables
3. **Inner Page Tables:** Each entry in the outer table points to a physical frame that holds a second-level (inner) page table.
4. **Final Mapping:** The inner table entry then provides the frame number in physical memory where the actual data page resides, combined with the final offset to get the physical address.

Example: Consider we have a process whose size is $4\text{ GB} = 2^{32}$

The process is now divided into a number of pages and we know that page size is equal to frame size = $4\text{ KB} = 2^{12}$

Now the logical address is represented as:



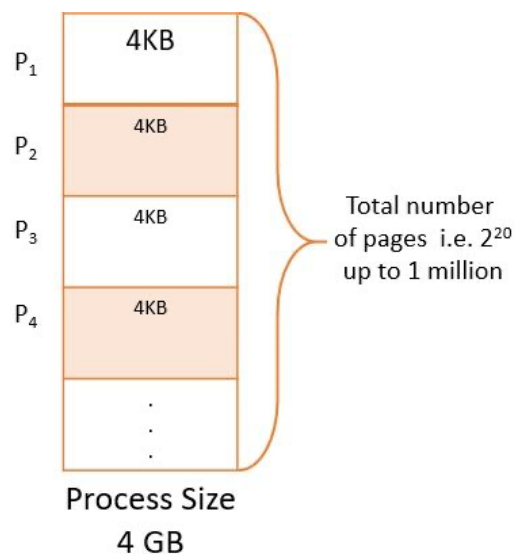
Logical address in bits = 32

Page size in bits = 12

Number of bits used to represent page number = $32 - 12 = 20$.

So the total number of the pages of the process would be = 2^{20} i.e. up to 1 million.

Now we have a process of size 4 GB which is divided into 1 million pages each of size 4KB.



Next, we have to implement a page table to store the information of pages of process i.e. which page is stored in which frame of the memory. As we have 1 million pages of the process there will be 1 million entries in the page table. Now in page table the page number provided by CPU in logical address act as an index which leads you to the frame number where this page is stored in main memory. This means each entry of the page table has the frame number.

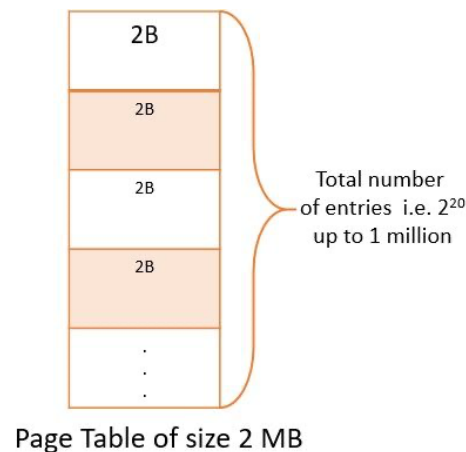
As we have seen above the frame number is represented by 17 bits so the size of each entry will be of 17 bits i.e. approx. 2 bytes.

Size of page table = number of entries * size of each entry

$$= 2^{20} * 2$$

$$= 2^{21}$$

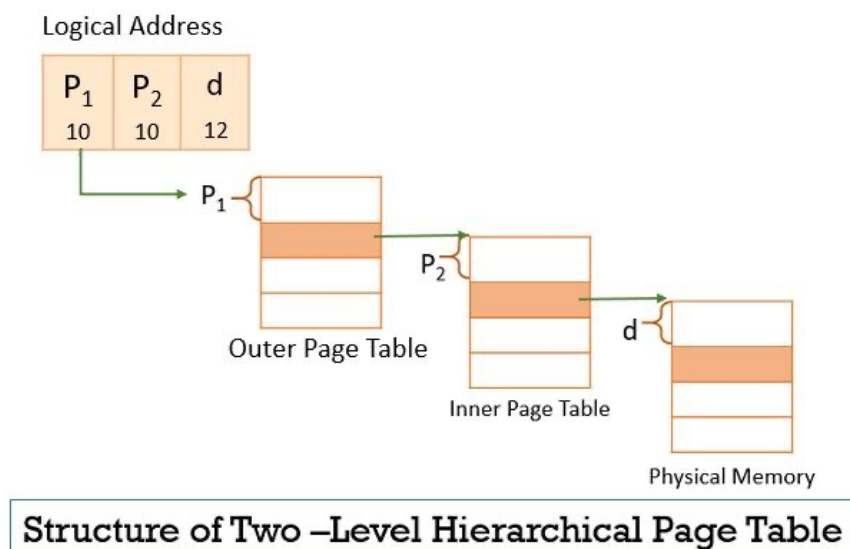
$$= 2 \text{ MB}$$



So can you store the page table of size 2 MB in a frame of the main memory where frame size is 4 KB? It is impossible.

So we need to divide the page table this division of page table can be accomplished in several ways. You can perform two-level division, three-level division on page table and so on.

Let us discuss two-level paging in which a page table itself is paged. So we will have two-page tables' inner page table and outer page table. We have a logical address with page number 20 and page offset 12. As we are paging a page table the page number will further get split to 10-bit page number and 10-bit offset



Here P_1 would act as an index and P_2 would act as an offset for the outer page table. Further, the P_2 would act as an index and d would act as an offset to inner page table to map the logical address of the page to the physical memory.

Hashed Page Tables

When the logical address space is beyond 32 bits in such case the page table is structured using hashed page table. Though we can structure the large page table using the multilevel page table it would consist of a number of levels increases the complexity of the page table.

The hashed page table is a convenient way to structure the page table where logical address space is beyond 32 bits. The hash table has several entries where each entry has a link list.

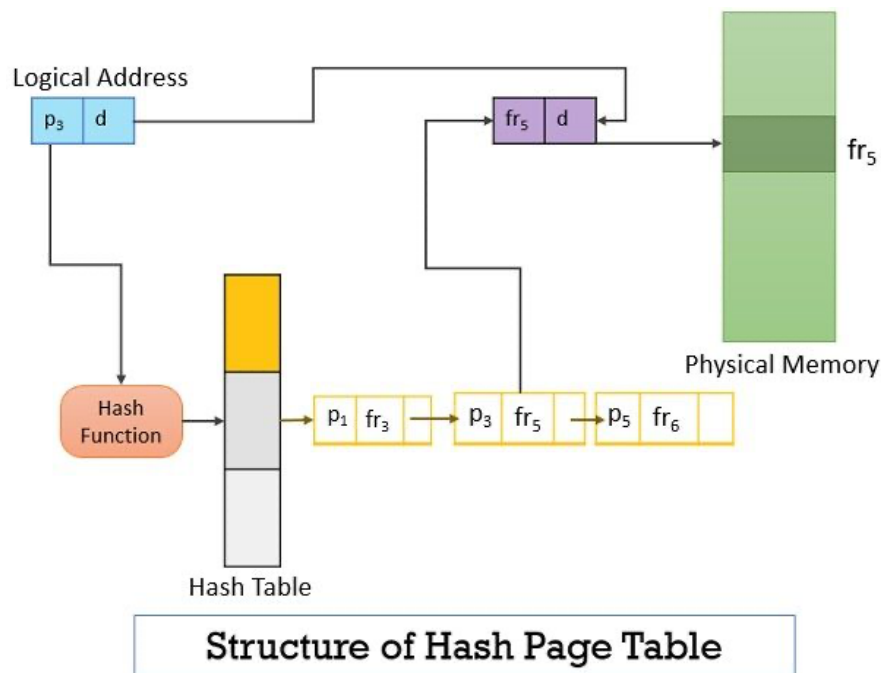
Each entry in hash table contains a linked list of elements that hash to the same location.

Each entry consists of;

- (a) Virtual page numbers
- (b) Value of mapped page frame.
- (c) Pointer to the next element in the linked list.

Working Procedure:

- The virtual page number in the virtual address is hashed into the hash table.
- Virtual page number is compared to field (a) in the 1st element in the linked list.
- If there is a match, the corresponding page frame (field (b)) is used to form the desired physical address.
- If there is no match, subsequent entries in the linked list are searched for a matching virtual page number.



The CPU generates a logical address for the page it needs. Now, this logical address needs to be mapped to the physical address. This logical address has two entries i.e. a page number (P_3 in our case) and an offset.

The page number from the logical address is directed to the hash function. The hash function produces a hash value corresponding to the page number. This hash value directs to an entry in the hash table. As we have studied earlier, each entry in the hash table has a link list. Here the page number is compared with the first element's first entry if a match is found then the second entry is checked.

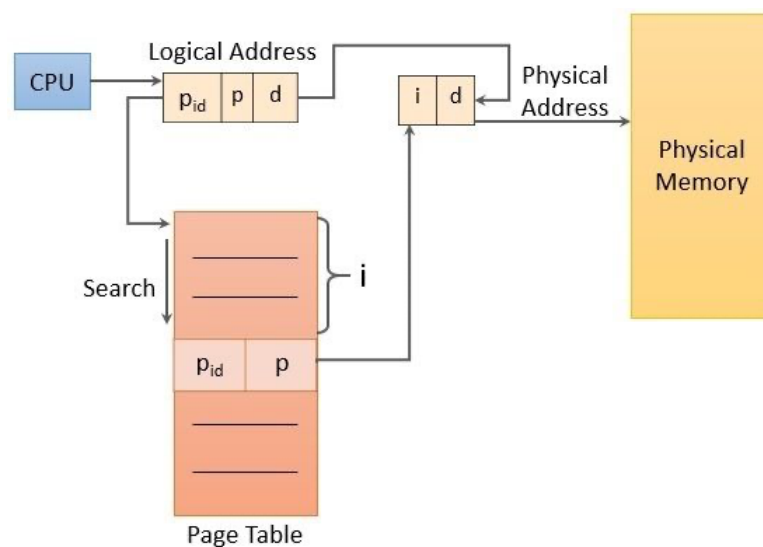
In the example above, the logical address includes page number P_3 which does not match the first element of the link list as it includes page number P_1 . So we will move ahead and check the next element; now this element has a page number entry i.e. P_3 so further we will check the frame entry of the element which is fr_5 . To this frame number, we will append the offset provided in the logical address to reach the page's physical address.

Inverted Page Table

The concept of normal paging says that every process maintains its own page table which includes the entries of all the pages belonging to the process. The large process may have a page table with millions of entries. Such a page table consumes a large amount of memory.

Consider we have six processes in execution. So, six processes will have some or the other of their page in the main memory which would compel their page tables also to be in the main memory consuming a lot of space. This is the drawback of the paging concept.

The inverted page table is the solution to this wastage of memory. The concept of an inverted page table involves the existence of single-page table which has entries of all the pages (may they belong to different processes) in main memory along with the information of the process to which they are associated. To get a better understanding consider the figure below of inverted page table.



Structure of Inverted Page Table

The CPU generates the logical address for the page it needs to access. This time the logical address consists of three entries process id, page number and the offset. The process id identifies the process, of which the page has been demanded, page number indicates which page of the process has been asked for and the offset value indicates the displacement required.

The match of process id along with associated page number is searched in the page table and say if the search is found at the i^{th} entry of page table then i and offset together generates the physical address for the requested page. This is how the logical address is mapped to a physical address using the inverted page table.

Though the inverted page table reduces the wastage of memory but it increases the search time. This is because the entries in an inverted page table are sorted on the basis of physical address

whereas the lookup is performed using logical address. It happens sometimes that the entire table is searched to find the match.

Shared Pages:

Shared pages in an operating system allow multiple processes to map different virtual addresses to the same physical memory frame, enabling them to share code or data. This technique reduces RAM usage, enhances performance, and facilitates inter-process communication by eliminating redundant copies of data, commonly used for shared libraries, IPC, or Copy-on-Write (CoW) mechanisms during `fork()`.

Key Aspects of Shared Pages:

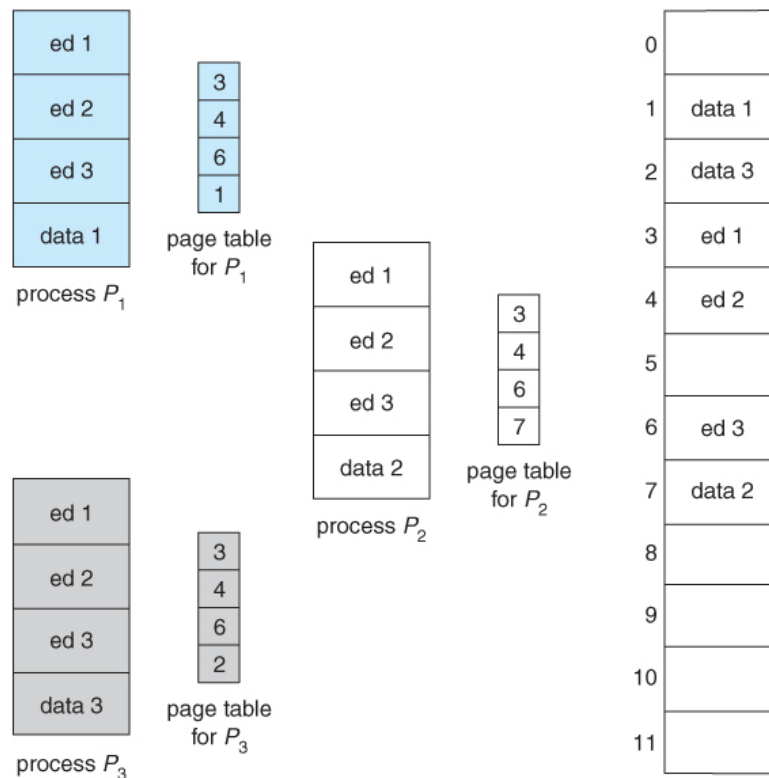
- **Memory Efficiency:** Instead of loading multiple instances of a shared library (e.g., `libc.so`) into RAM, the OS loads it once into a physical frame and maps it to the address space of all requesting processes.
- **Re-entrant Code:** Shared code must be re-entrant or pure, meaning it cannot modify itself while running. This allows multiple processes to execute the same code simultaneously without interfering with each other.
- **Copy-on-Write (CoW):** When a process forks, parent and child share pages initially. If either modifies a page, the OS creates a copy for that process, allowing efficient process creation without immediate, massive memory duplication.
- **Virtual Mapping:** Shared pages do not have to exist at the same virtual address in each process's address space.
- **Security Risks:** If not properly managed, shared pages can lead to security vulnerabilities if one process modifies data meant to be read-only for another.

Advantages of Shared Pages:

- **Reduced Memory Usage:** Less physical memory is used, leaving more space for other processes.
- **Performance Improvement:** Lower memory overhead and faster process startup times (especially with `fork`).
- **Improved Communication:** Processes can share data without using complex inter-process communication mechanisms.

Disadvantages:

- **Security Risks:** Unauthorized access or modification of sensitive data, if permissions are not properly set.
- **Complexity:** Requires careful management by the operating system to handle reference counting (tracking how many processes use a page).



Drawbacks of Paging: Paging in operating systems primarily causes internal fragmentation, as fixed-size pages may leave unused space at the end of a process. Key drawbacks include increased memory access time (due to two memory accesses—one for the page table, one for data), overhead from managing page tables, and potential page faults and Complexity in Hardware (Address translation necessitates specialized hardware (like the Memory Management Unit)).

Example:

In the worst case a process would need n pages plus one byte. It would be allocated $n+1$ frames resulting in an internal fragmentation of almost an entire frame. Let's say

Page size = 2048 bytes

Process size = 72766 bytes

Process needs 35 pages plus 1086 bytes.

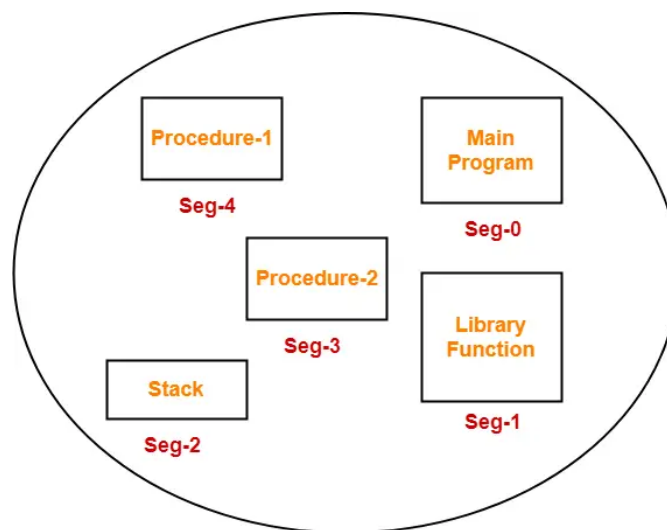
It is allocated 36 frames resulting in an internal fragmentation of 962 bytes.

Segmentation

Below is a **fully bridged, coherent, examination-ready 50-mark answer on Segmentation**, written in **continuous textbook style**, covering **every minute concept**, **all formulae**, **hardware details**, **advantages**, **disadvantages**, **comparisons**, and **combined schemes**. You can reproduce this directly in an exam.

SEGMENTATION IN OPERATING SYSTEM

Segmentation is a **memory-management technique** used by an operating system in which a process is divided into a number of **logical, variable-sized parts called segments**, and each segment is stored separately in physical memory. Unlike contiguous memory allocation, where a process occupies one continuous block of memory, segmentation reflects the **logical structure of a program** and allows each part of the program to be treated independently.



A segment represents a **meaningful unit** of the program such as:

- main program (code)
- procedures or functions
- global and local variables (data)
- stack
- heap

- large arrays or tables

Thus, segmentation matches the **programmer's view of memory** rather than treating memory as a single linear array of bytes.

Logical Address Structure in Segmentation

In segmentation, the CPU does **not** generate a single linear address. Instead, every logical address consists of **two parts**:

$$\text{Logical Address} = \langle \text{segment number, offset} \rangle$$

- **Segment number (s)**: identifies which segment is being referenced
- **Offset (d)**: specifies the displacement within that segment

Example:

- $\langle 0, x \rangle \rightarrow$ code segment
- $\langle 1, y \rangle \rightarrow$ data segment
- $\langle 2, z \rangle \rightarrow$ stack segment

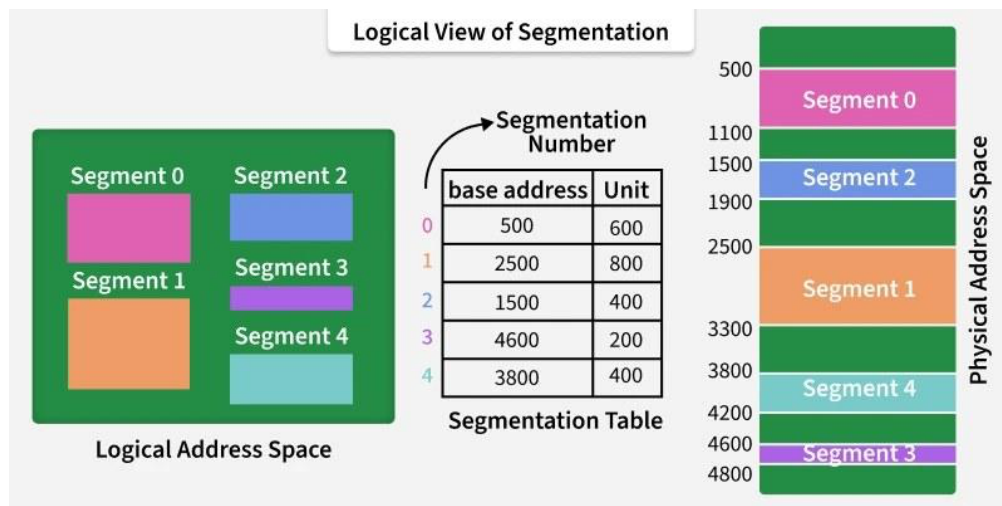
An address $\langle 1, 50 \rangle$ means *50 bytes from the beginning of segment 1*.

This structure ensures that **every memory reference belongs to a specific logical segment**, enabling fine-grained protection and sharing.

Segment Table

For each process, the operating system maintains a **segment table**. The segment table contains **one entry per segment** of the process. Each segment table entry consists of:

1. **Base address (Base)**: The starting physical address of the segment in main memory.
2. **Limit (Length)**: The size of the segment in bytes.
3. **Protection bits (optional but common)**: Specify permitted operations:
 - Read (R)
 - Write (W)
 - Execute (X)
4. **Valid/Invalid bit (in some systems)**: Indicates whether the segment exists in memory.



The segment table itself is stored in memory, and the CPU maintains a **Segment Table Base Register (STBR)** that points to the start of the current process's segment table.

Address Translation in Segmentation

Address translation is performed by the **Memory Management Unit (MMU)** in hardware.

Step-by-step Translation

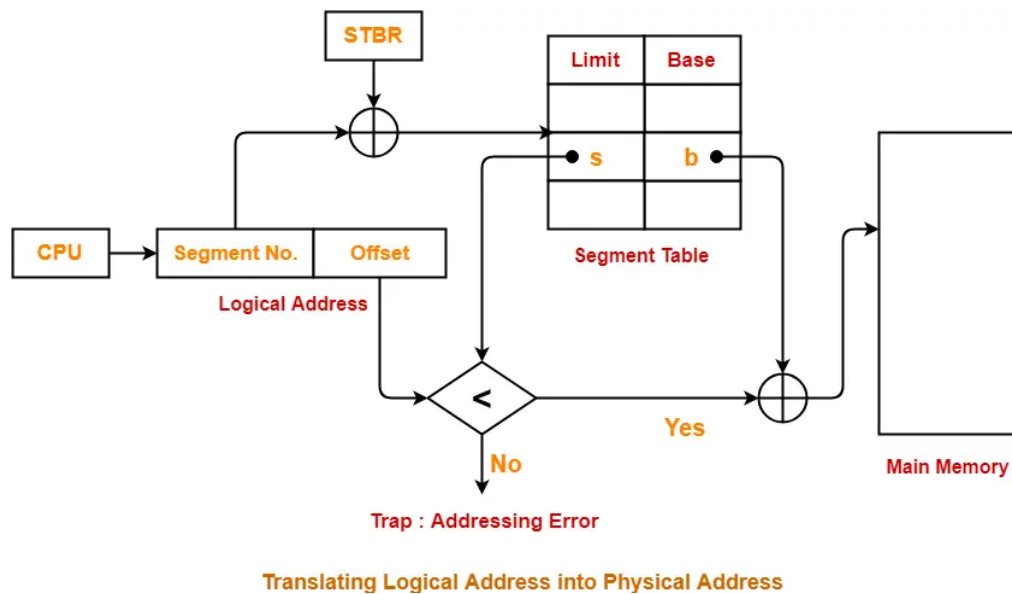
Given a logical address $\langle s, d \rangle$:

1. **Segment number check:** Verify that segment number s is valid for the process.
2. **Segment table lookup:** Use s as an index into the segment table to fetch:
 - base(s)
 - limit(s)
 - protection bits
3. **Bounds check:** Ensure: $d < \text{limit}$. If this condition fails, a **segmentation fault** occurs.
4. **Protection check:** Verify that the requested operation (read/write/execute) is permitted.

Physical address calculation

$$\text{Physical Address} = \text{Base} + \text{Offset}$$

This formula is central to segmentation and simultaneously enforces **address translation** and **memory protection**.



Numerical Example

Assume a process has the following segment table:

Segment	Base	Limit
0 (Code)	2000	600
1 (Data)	8000	1000
2 (Stack)	12000	400

Example 1

Logical address $\langle 1, 350 \rangle$:

- $350 < 1000 \rightarrow$ valid
- Physical address = $8000 + 350 = \mathbf{8350}$

Example 2

Logical address $\langle 2, 500 \rangle$:

- $500 \geq 400 \rightarrow$ invalid
- **Segmentation fault occurs**

Protection Mechanism in Segmentation

Segmentation provides **strong memory protection** because:

- Each segment has its own protection bits
- Illegal memory accesses are detected in hardware

Typical protections:

- **Code segment:** Read + Execute (no write)
- **Data segment:** Read + Write (no execute)
- **Stack segment:** Read + Write

Any violation causes a **trap to the operating system**, preventing memory corruption.

Sharing in Segmentation

Segmentation naturally supports **memory sharing**.

- Multiple processes can share the **same physical segment**
- Segment table entries of different processes can point to the same base address

Examples:

- Shared libraries (read-only, executable)
- Shared data structures (read-write)

This improves:

- Memory utilization
- Inter-process communication
- System efficiency

Advantages of Segmentation

1. Matches the **logical structure of programs**
2. Supports **fine-grained protection**
3. Enables **efficient sharing**

4. No internal fragmentation
5. Flexible memory allocation
6. Easier debugging and modular programming

Disadvantages of Segmentation

External Fragmentation

Segments are **variable sized**, leading to:

- Free memory split into small holes
- Allocation failure even when total free memory is sufficient

Compaction

To reduce fragmentation, the OS may perform **compaction**:

- Move segments to combine holes
- Update base addresses in segment table

However, compaction is:

- Time-consuming
- Requires relocation support
- Not practical for frequent use

Hardware Support for Segmentation

Segmentation requires:

- MMU support
- Segment registers
- Fast bounds and protection checks

Example:

- **Intel x86 architecture** uses segment selectors and descriptors

Pure software segmentation (used by compilers/linkers) does **not** provide hardware protection or automatic translation.

Segmentation vs Paging

Feature	Segmentation	Paging
Unit size	Variable	Fixed
Logical meaning	Yes	No
Fragmentation	External	Internal
Programmer view	Visible	Invisible
Protection	Per segment	Per page

Segmentation with Paging

To combine advantages of both:

- Each segment is divided into pages
- Segment table entry points to a **page table**

Segmentation with paging is a hybrid memory-management technique that combines the logical view and protection of segmentation with the efficient physical memory utilization of paging.

In this scheme:

- A process is **logically divided into segments** (code, data, stack, etc.).
- Each segment is **further divided into fixed-size pages**.
- Physical memory is divided into **fixed-size frames**.
- Pages of segments are mapped to frames using **page tables**.

Thus, segmentation is used to define what memory means, and paging is used to decide where it is placed in physical memory.

Why Segmentation with Paging Is Needed: Pure segmentation suffers from **external fragmentation** because segments are variable-sized. Pure paging avoids external fragmentation but **loses logical structure** and fine-grained protection.

Segmentation with paging solves both problems by:

- Keeping **logical units (segments)** for protection and sharing

- Using **fixed-size pages** inside each segment to avoid external fragmentation

Logical Address Structure

In segmentation with paging, the logical address has **three parts**:

$\langle \text{Segment Number, Page Number, Page Offset} \rangle$

- **Segment number (s)**: identifies the logical segment
- **Page number (p)**: identifies the page within that segment
- **Page offset (d)**: identifies the exact byte within the page

This is an extension of pure segmentation, where the offset is now split into page number and offset.

Data Structures Used

1. Segment Table: Each process has a segment table. Each entry contains:

- Segment length (limit)
- Base address of the page table for that segment
- Protection bits (R/W/X)

2. Page Table (Per Segment): Each segment has its own page table. Each page table entry contains:

- Frame number
- Valid/invalid bit
- Optional protection and status bits

Address Translation Mechanism (Step-by-Step)

Given a logical address $\langle s, p, d \rangle$:

1. Segment number check
 - Use s to index the segment table
 - Check whether segment s is valid
2. Segment limit check
 - Verify that the logical address is within the segment length
3. Locate page table

- Segment table entry gives the base address of the page table for segment s
4. Page table lookup
 - Use page number p to index that page table
 - Retrieve the corresponding frame number
 5. Frame offset calculation
 - Combine frame number and page offset d to form physical address
- $$\boxed{\text{Physical Address} = (\text{Frame Number} \times \text{Page Size}) + \text{Offset}}$$
6. Protection and validity checks
 - If any check fails $\rightarrow$ trap to OS

Segmentation with paging uses segment descriptors. Those descriptors **may be stored in GDT/LDT (on x86)**. Paging is applied **after segmentation**, independently.

What is GDT / LDT?

The **Global Descriptor Table (GDT)** and **Local Descriptor Table (LDT)** are **x86-specific hardware structures** used to store **segment descriptors**.

A **segment descriptor** contains:

- Base address of the segment
- Segment limit
- Access rights (R/W/X)
- Privilege level
- Segment type

These descriptors define **how segmentation works in x86 systems**.

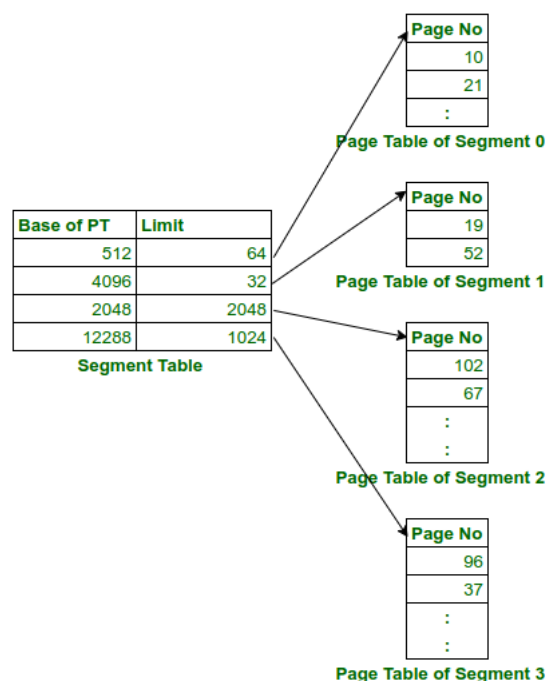
The Global Descriptor Table (**GDT**) and Local Descriptor Table (**LDT**) are hardware-specific data structures used in segmentation, particularly in x86 architecture, to store segment descriptors containing base address, limit, and access rights. They are part of the segmentation mechanism, not paging. In systems that use segmentation with paging,

segmentation is performed first using a segment table (which may be implemented as GDT/LDT in x86), producing a linear address. Paging is then applied to translate the linear address into a physical address. Therefore, GDT/LDT are not separate or mandatory concepts of segmentation with paging, but are architecture-dependent implementations of segmentation.

Benefits of Segmentation with Paging:

- Logical structure of segmentation
- Reduced external fragmentation due to paging

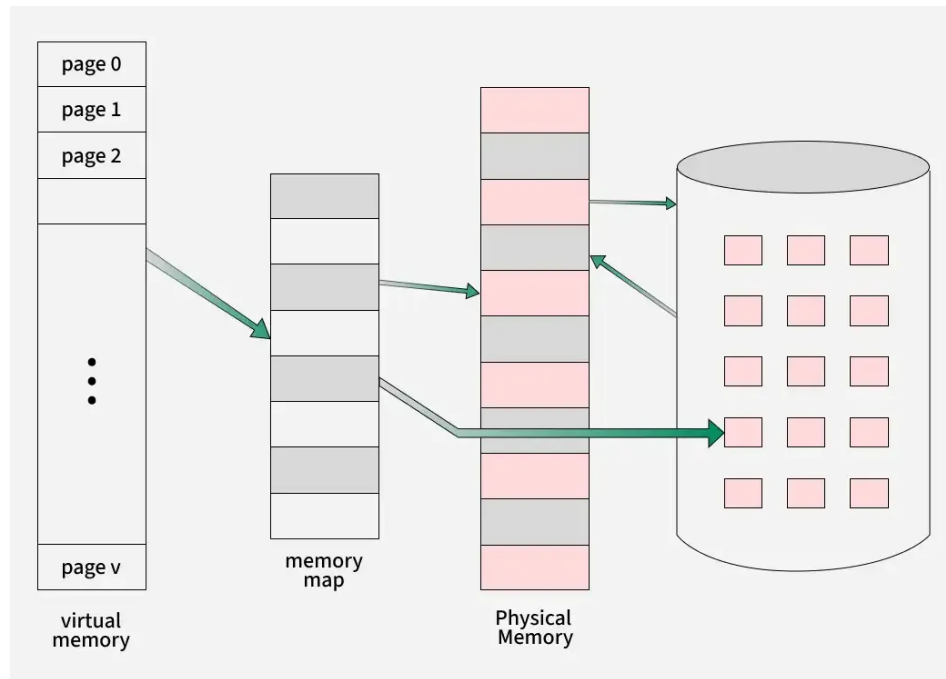
This scheme is widely used in modern systems.



Segmentation is a powerful memory-management technique that divides a process into logical units and maps them independently into physical memory. It provides strong protection, efficient sharing, and a natural programming model using a two-part logical address (segment, offset) and the translation formula **physical address = base + offset**. While segmentation suffers from external fragmentation, techniques like compaction and segmentation with paging mitigate its limitations. Understanding segmentation, its translation mechanism, protection features, and comparison with paging is essential for mastering operating-system memory management.

VIRTUAL MEMORY

Virtual memory is a **memory-management technique** that allows a computer system to execute programs that are **larger than the available physical main memory (RAM)** by using secondary storage (usually a hard disk or SSD) as an extension of main memory.



virtual memory in OS

In a virtual memory system:

- Each process is given the illusion that it has a **large, contiguous address space**
- Only the **currently needed portions** of the program are kept in RAM
- The remaining parts are stored on disk and are brought into memory **on demand**

Thus, virtual memory separates:

- **Logical address space** (what the program sees)
- **Physical address space** (actual RAM)

This abstraction greatly improves **memory utilization, multiprogramming, and system performance**.

Need for Virtual Memory

Virtual memory is required because:

1. Programs are often **larger than physical memory**
2. Not all parts of a program are needed at the same time
3. Keeping entire programs in RAM wastes memory
4. Better CPU utilization requires more processes in memory

Without virtual memory:

- Program size would be limited by RAM size
- Multiprogramming would be poor
- Memory utilization would be inefficient

The key idea of virtual memory is based on **locality of reference**, which states that:

- Programs tend to access **only a small portion of their address space at any time**

Types of locality:

- **Temporal locality**: recently used data is likely to be used again
- **Spatial locality**: nearby memory locations are likely to be accessed

Virtual memory exploits this by loading only the **working set** of a process into RAM.

Virtual Address Space

The **virtual address space** of a process is the set of all logical addresses that the process can generate.

Characteristics:

- Appears large and contiguous
- Independent of physical memory size
- Managed by the operating system and MMU

Each virtual address must be translated into a physical address before memory access.

Paging as the Basis of Virtual Memory

Virtual memory is most commonly implemented using **paging**.

Paging Recap

- Virtual address space → divided into **pages**
- Physical memory → divided into **frames**
- Page size = Frame size (e.g., 4 KB)

Address Translation in Virtual Memory

A virtual address is divided as:

$$\boxed{\text{Virtual Address} = \langle \text{Page Number}, \text{Offset} \rangle}$$

Translation steps:

1. Page number indexes the **page table**
2. Page table entry gives:
 - Frame number
 - Valid/invalid bit
3. Physical address is computed as:

$$\boxed{\text{Physical Address} = (\text{Frame Number} \times \text{Page Size}) + \text{Offset}}$$

Demand Paging (Core of Virtual Memory)

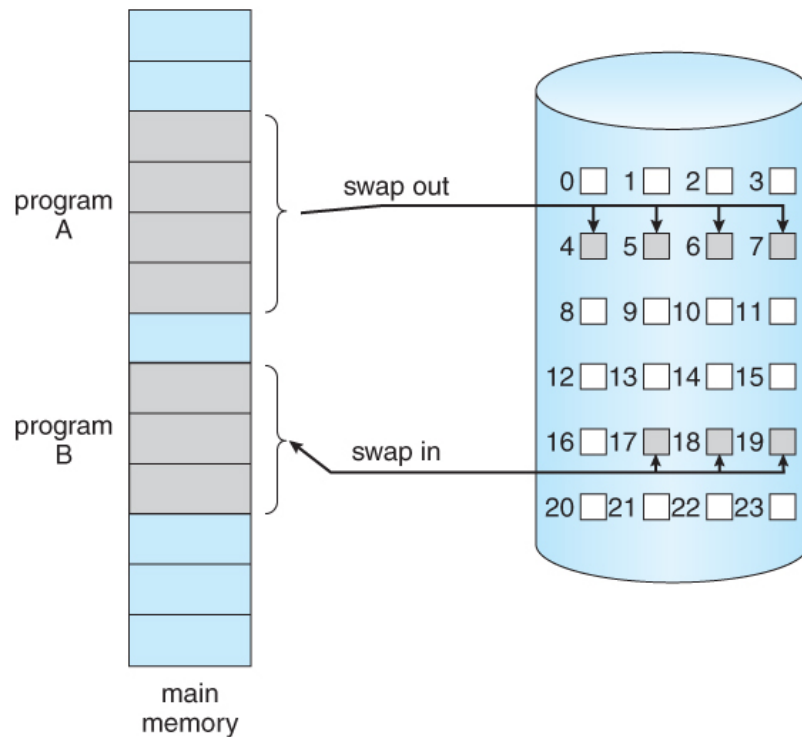
Demand paging is a virtual memory technique in which pages are loaded into RAM **only when they are referenced**.

Initially:

- Page table entries are marked invalid
- No pages are loaded into memory

Demand Paging is similar to a paging system with swapping.

- Demand Paging - Bring a page into memory only when it is needed
- To execute a process, swap that entire process into memory. Rather than swapping the entire process into memory however, we use —Lazy Swapper
- **Lazy Swapper** - Never swaps a page into memory unless that page will be needed.



Transfer of a paged memory to contiguous disk space

Basic Concepts: Instead of swapping in the whole processes, the pager brings only those necessary pages into memory. Thus,

1. It avoids reading into memory pages that will not be used anyway.
2. Reduce the swap time.
3. Reduce the amount of physical memory needed.

To differentiate between those pages that are in memory & those that are on the disk we use the **Valid-Invalid bit**.

Valid-Invalid bit

A valid – invalid bit is associated with each page table entry.

Valid bit is associated when page is in memory.

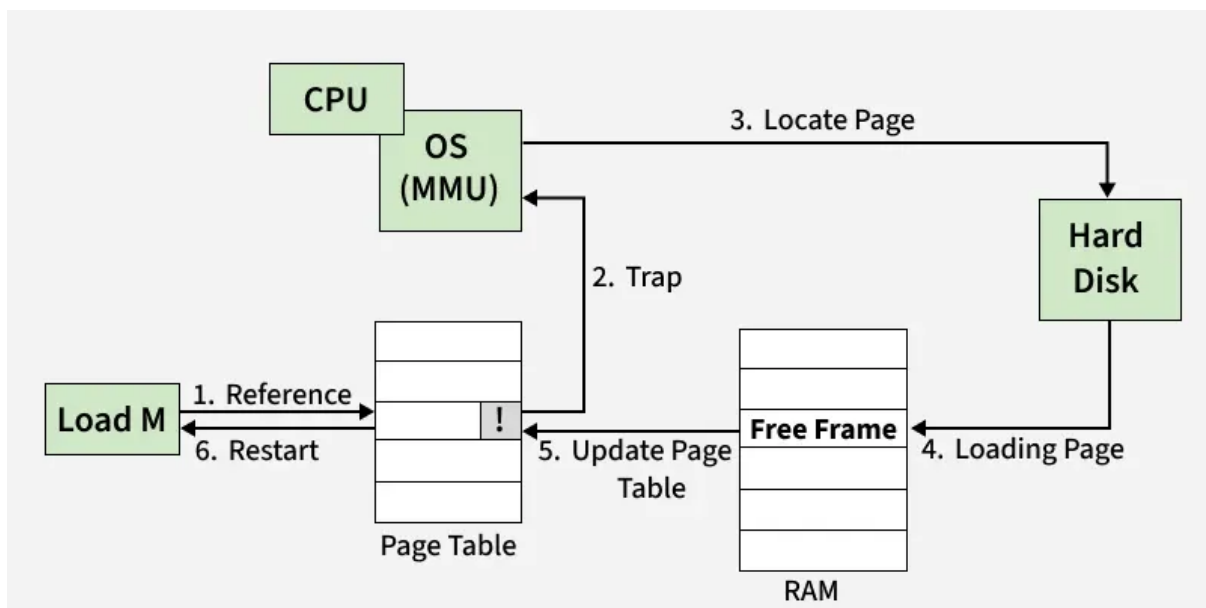
In-Valid associated with:

- Invalid page
- Valid page but is currently on the disk

Page Fault: A page fault occurs when a process accesses a page that is not currently present in physical memory.

Page Fault Handling Steps

1. Hardware detects invalid page reference
2. Trap to OS(a controlled, automatic transfer of execution from a user program to the operating system, caused by an exception detected by the hardware. Occurs when the CPU detects that a referenced page is not present in main memory.)
3. OS checks if reference is legal
4. Finds a free frame (or selects victim page)
5. Reads page from disk into frame
6. Updates page table
7. Restarts instruction



Effective Access Time (EAT): Performance of virtual memory is measured using **Effective Access Time**.

Let:

- m = memory access time
- p = page fault rate
- s = page fault service time

$$\boxed{EAT = (1 - p) \times m + p \times s}$$

Since disk access is slow, **page fault rate must be very small.**

Pure demand paging

- Never bring a page into memory until it is required. We could start a process with no pages in memory.
- When the OS sets the instruction pointer to the 1st instruction of the process, which is on the non-memory resident page, then the process immediately faults for the page.
- After this page is brought into the memory, the process continues to execute, faulting as necessary until every page that it needs is in memory.

Performance of demand paging

o Let p be the probability of a page fault $0 \leq p \leq 1$

o Effective Access Time (EAT)

$$\text{EAT} = (1 - p) \times m_a + p \times \text{page fault time.}$$

Where m_a $\diamond$ memory access, p $\diamond$ Probability of page fault ($0 \leq p \leq 1$)

o The memory access time denoted m_a is in the range 10 to 200 ns. o

If there are no page faults then $\text{EAT} = m_a$.

o To compute effective access time, we must know how much time is needed to service a page fault.

- A page fault causes the following sequence to occur:
 1. Trap to the OS
 2. Save the user registers and process state.
 3. Determine that the interrupt was a page fault.
 4. Check whether the reference was legal and find the location of page on disk.
 5. Read the page from disk to free frame.
 - a. Wait in a queue until read request is serviced.
 - b. Wait for seek time and latency time.
 - c. Transfer the page from disk to free frame.
 6. While waiting, allocate CPU to some other user.
 7. Interrupt from disk.

8. Save registers and process state for other users.
9. Determine that the interrupt was from disk.
10. Reset the page table to indicate that the page is now in memory.
11. Wait for CPU to be allocated to this process again.
12. Restart the instruction that was interrupted .

Process Creation: Virtual memory enhances the performance of creating and running processes:

- Copy-on-Write
- Memory-Mapped Files

a) Copy-on-Write:

- fork() creates a child process as a duplicate of the parent process & it worked by creating copy of the parent address space for child, duplicating the pages belonging to the parent.
- Copy-on-Write (COW) allows both parent and child processes to initially *share* the same pages in memory. These shared pages are marked as Copy-on-Write pages, meaning that if either process modifies a shared page, a copy of the shared page is created.
- vfork(): With this the parent process is suspended & the child process uses the address space of the parent.
 - Because vfork() does not use Copy-on-Write, if the child process changes any pages of the parent's address space, the altered pages will be visible to the parent once it resumes.
 - Therefore, vfork() must be used with caution, ensuring that the child process does not modify the address space of the parent.

(b)Memory – mapped files:

- Sequential read of a file on disk uses open() , read() and write()
- Every time a file is accessed it requires a system call and disk access.
- Alternative method: "Memory – mapped files"
 - Allowing a part of virtual address space to be logically associated with file
 - Mapping a disk block to a page in memory.

Page Replacement Algorithms: When no free frame is available, OS must replace a page. If no frames are free, we could find one that is not currently being used & free it.

- We can free a frame by writing its contents to swap space & changing the page table to indicate that the page is no longer in memory.
- Then we can use that freed frame to hold the page for which the process faulted.

Basic Page Replacement

1. Find the location of the desired page on disk
2. Find a free frame
 - i. If there is a free frame , then use it.
 - ii. If there is no free frame, use a page replacement algorithm to select a **victim** frame
 - iii. Write the victim page to the disk, change the page & frame tables accordingly.
3. Read the desired page into the (new) free frame. Update the page and frame tables.
4. Restart the process

If no frames are free, two page transfers are required & this situation effectively doubles the page- fault service time.

Modify (dirty) bit:

- It indicates that any word or byte in the page is modified.
- When we select a page for replacement, we examine its modify bit.
 - If the bit is set, we know that the page has been modified & in this case we must write that page to the disk.
 - If the bit is not set, then if the copy of the page on the disk has not been overwritten, then we can avoid writing the memory page on the disk as it is already there.

1. FIFO (First In First Out): This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

Example 1: Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3-page frames. Find the number of page faults using FIFO Page Replacement Algorithm.

Page
reference

1, 3, 0, 3, 5, 6, 3

1	3	0	3	5	6	3
		0	0	0	0	3
	3	3	3	3	6	6
1	1	1	1	5	5	5
Miss	Miss	Miss	Hit	Miss	Miss	Miss

Total Page Fault = 6

FIFO - Page Replacement

- Initially, all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots ---> 3 Page Faults.
- When 3 comes, it is already in memory so ---> 0 Page Faults.
- Then 5 comes, it is not available in memory, so it replaces the oldest page slot i.e 1. ---> 1 Page Fault.
- 6 comes, it is also not available in memory, so it replaces the oldest page slot i.e 3 ---> 1 Page Fault.
- Finally, when 3 come it is not available, so it replaces 0 1-page fault.

2. Optimal Page Replacement: In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.

Example: Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frame. Find number of page fault using Optimal Page Replacement Algorithm.

Page
reference

7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Optimal Page Replacement

- Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots ---> 4 Page faults
- 0 is already there so ---> 0 Page fault. when 3 came it will take the place of 7 because it is not used for the longest duration of time in the future---> 1 Page fault.
- 0 is already there so ---> 0 Page fault. 4 will takes place of 1 ---> 1 Page Fault.

Now for the further page reference string ---> 0 Page fault because they are already available in the memory. Optimal page replacement is perfect, but not possible in practice as the operating system cannot know future requests. The use of Optimal Page replacement is to set up a benchmark so that other replacement algorithms can be analysed against it.

3. Least Recently Used: In this algorithm, page will be replaced which is least recently used.

Example: Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page frames. Find number of page faults using LRU Page Replacement Algorithm.

Page
reference

7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Here LRU has same number of page fault as optimal but it may differ according to question.

Least Recently Used - Page Replacement

- Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots ---> 4 Page faults.
- 0 is already there so ---> 0 Page fault. when 3 came it will take the place of 7 because it is least recently used ---> 1 Page fault.
- 0 is already in memory so ---> 0 Page fault.
- 4 will takes place of 1 ---> 1 Page Fault.
- Now for the further page reference string ---> 0 Page fault because they are already available in the memory.

4. Clock / Second Chance: The Second Chance Page Replacement Algorithm is an improvement over FIFO (First-In, First-Out). FIFO may remove frequently used pages, but Second Chance reduces this problem by using a reference bit.

- When a page is accessed, its reference bit is set to 1.
- During replacement, the algorithm checks pages in order:
 - If a page's bit = 0, it is replaced.
 - If the bit = 1, it is reset to 0 and the page gets a "second chance," moving to the back of the queue.

This way, frequently used pages are less likely to be removed. Traditionally, Second Chance and its variant, Clock, were considered less efficient than LRU (Least Recently Used).

However, recent research shows that Second Chance and Clock can achieve a lower miss ratio and offer better scalability, making them more efficient than LRU in modern systems.

The algorithm works by associating a reference bit with each page in memory:

- Reference Bit = 1, The page has been recently used.
- Reference Bit = 0, The page has not been recently used.

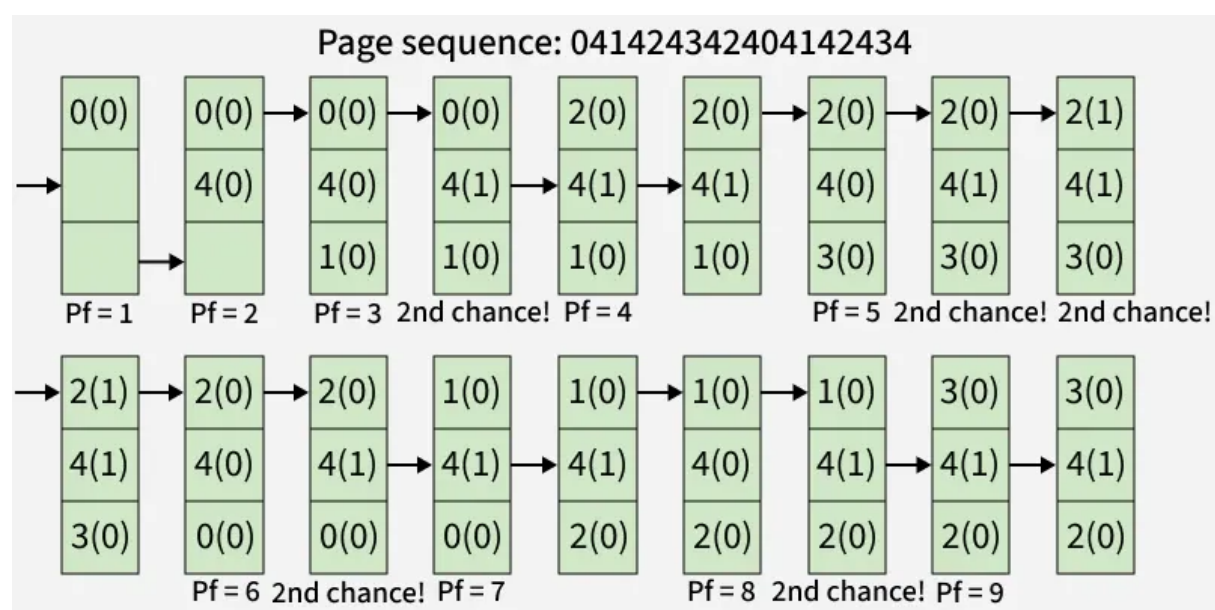
Steps:

1. Pages are maintained in a queue (like FIFO).
2. When a page fault occurs, the algorithm checks the page at the front of the queue.
3. If the reference bit = 0, the page is replaced.
4. If the reference bit = 1:
 - The reference bit is reset to 0.
 - The page is moved to the back of the queue (given a second chance).
 - The search continues until a page with reference bit = 0 is found.

Thus, frequently accessed pages are less likely to be replaced.

Example

Let's say the reference string is **0 4 1 4 2 4 3 4 2 4 0 4 1 4 2 4 3 4** and we have **3** frames. Let's see how the algorithm proceeds by tracking the second chance bit and the pointer.



Page Replacement

- Initially, all frames are empty so after the first 3 passes they will be filled with {0, 4, 1} and the second chance array will be {0, 0, 0} as none has been referenced yet. Also, the pointer will cycle back to 0.
- Pass-4:** Frame={0, 4, 1}, second_chance = {0, 1, 0} [4 will get a second chance], pointer = 0 (No page needed to be updated so the candidate is still page in frame 0), pf = 3 (No increase in page fault number).
- Pass-5:** Frame={2, 4, 1}, second_chance= {0, 1, 0} [0 replaced; it's second chance bit was 0, so it didn't get a second chance], pointer=1 (updated), pf=4
- Pass-6:** Frame={2, 4, 1}, second_chance={0, 1, 0}, pointer=1, pf=4 (No change)
- Pass-7:** Frame={2, 4, 3}, second_chance= {0, 0, 0} [4 survived but it's second chance bit became 0], pointer=0 (as element at index 2 was finally replaced), pf=5
- Pass-8:** Frame={2, 4, 3}, second_chance= {0, 1, 0} [4 referenced again], pointer=0, pf=5
- Pass-9:** Frame={2, 4, 3}, second_chance= {1, 1, 0} [2 referenced again], pointer=0, pf=5
- Pass-10:** Frame={2, 4, 3}, second_chance= {1, 1, 0}, pointer=0, pf=5 (no change)
- Pass-11:** Frame={2, 4, 0}, second_chance= {0, 0, 0}, pointer=0, pf=6 (2 and 4 got second chances)
- Pass-12:** Frame={2, 4, 0}, second_chance= {0, 1, 0}, pointer=0, pf=6 (4 will again get a second chance)
- Pass-13:** Frame={1, 4, 0}, second_chance= {0, 1, 0}, pointer=1, pf=7 (pointer updated, pf updated)
- Page-14:** Frame={1, 4, 0}, second_chance= {0, 1, 0}, pointer=1, pf=7 (No change)
- Page-15:** Frame={1, 4, 2}, second_chance= {0, 0, 0}, pointer=0, pf=8 (4 survived again due to 2nd chance!)
- Page-16:** Frame={1, 4, 2}, second_chance= {0, 1, 0}, pointer=0, pf=8 (2nd chance updated)

- **Page-17:** Frame={3, 4, 2}, second_chance= {0, 1, 0}, pointer=1, pf=9 (pointer, pf updated)
- **Page-18:** Frame={3, 4, 2}, second_chance= {0, 1, 0}, pointer=1, pf=9 (No change)

Comparison of Page Replacement Algorithms (Advantages & Disadvantages)

Algorithm	Advantages (Detailed)	Disadvantages (Detailed)
FIFO (First-In First-Out)	FIFO is very simple to understand and implement, as it only requires maintaining a queue of pages in the order in which they were loaded into memory. It has low overhead, since no additional information about page usage or frequency needs to be tracked. Because of its simplicity, FIFO is useful for explaining the basic concept of page replacement and is easy to implement in systems where complexity must be minimised.	FIFO makes replacement decisions without considering actual page usage, so it may remove pages that are heavily used simply because they are old. This can lead to poor performance and a high number of page faults. FIFO also suffers from Belady's anomaly, where increasing the number of frames can paradoxically increase the number of page faults. Due to these issues, FIFO is rarely used in modern operating systems.
Optimal (OPT)	The Optimal algorithm always replaces the page that will not be used for the longest time in the future, which guarantees the minimum possible number of page faults for a given reference string. Because of this property, it serves as a theoretical benchmark against which other page replacement algorithms are compared. It helps in evaluating how close a practical algorithm comes to ideal performance.	The major disadvantage of the Optimal algorithm is that it cannot be implemented in real systems, as it requires exact knowledge of future memory references, which is impossible during program execution. Therefore, it is used only for theoretical analysis and simulation, not in actual operating systems.
LRU (Least Recently Used)	LRU replaces the page that has not been used for the longest time in the past, which aligns well with the principle of locality of reference. As a result, LRU generally produces far fewer page faults than FIFO and provides performance close to the Optimal algorithm. It also does not suffer from Belady's anomaly, making its behaviour more predictable as the number of frames increases.	The main disadvantage of LRU is that it is expensive to implement exactly, as the system must track the order of page accesses. Implementations using timestamps or stacks require additional hardware support or high overhead, increasing memory usage and slowing down the system. Because of this complexity, true LRU is rarely implemented directly in practice.

Clock (Second Chance)	The Clock algorithm is a practical and efficient approximation of LRU. It uses a reference bit to give recently used pages a “second chance,” significantly improving performance over FIFO. The algorithm is easy to implement, requires less overhead than LRU, and performs well in real systems. Due to its balance between simplicity and efficiency, it is widely used in modern operating systems.	Although the Clock algorithm improves upon FIFO, it is not as accurate as true LRU, since it only distinguishes between recently used and not recently used pages. Some pages may receive multiple second chances and remain in memory longer than necessary, leading to suboptimal replacement decisions compared to LRU or Optimal.
-----------------------------	--	--

Thrashing: Thrashing occurs when a system spends more time swapping pages in and out of memory than executing actual process instructions. In other words, the CPU is almost idle because it is constantly handling page faults instead of doing productive work.

It is a severe performance problem in demand-paged virtual memory systems.

Causes of the issue:

- High paging activity is called **thrashing**.
- If a process does not have —enough‖ pages, the page-fault rate is very high. This leads to:
 - low CPU utilization
 - operating system thinks that it needs to increase the degree of multiprogramming
 - another process is added to the system
- When the CPU utilization is low, the OS increases the degree of multiprogramming.
- If global replacement is used then as processes enter the main memory they tend to steal frames belonging to other processes.
- Eventually all processes will not have enough frames and hence the page fault rate becomes very high.
- Thus swapping in and swapping out of pages only takes place.

Effects:

- CPU utilization drops
- System performance collapses

Solutions to Thrashing

1. Working Set Model

It is based on the assumption of the model of locality.

- Locality is defined as the set of pages actively used together.
- Working set is the set of pages in the most recent Δ page references
- Δ is the working set window.
 - if Δ too small , it will not encompass entire locality
 - if Δ too large ,it will encompass several localities
 - if $\Delta = \infty \Rightarrow$ it will encompass entire program
- $D = \sum WSS_i$
 - Where WSS_i is the working set size for process i .
 - D is the total demand of frames
- if $D > m$ then Thrashing will occur.

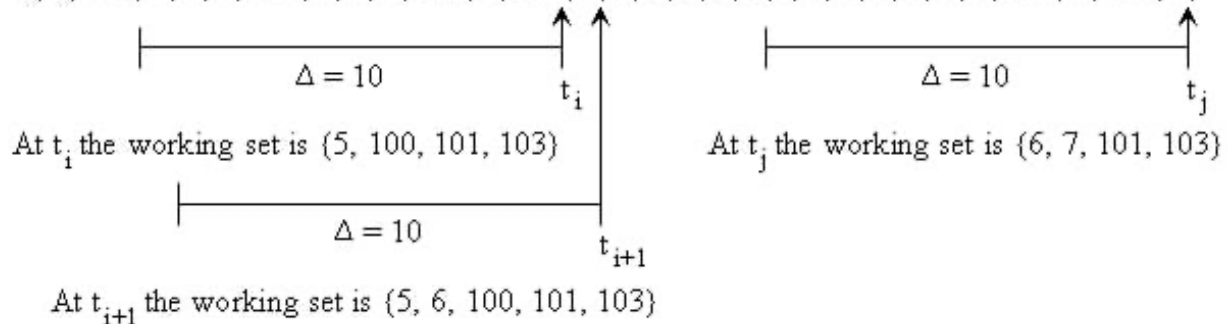
Working-Set Example

Suppose that $\Delta = 10$ (unrealistically small)

Consider the working set at two different times, t_i and t_j

Reference string:

5, 5, 101, 5, 5, 5, 101, 5, 5, 5, 100, 5, 103, 6, 6, 6, 6, 6, 6, 6, 7, 7, 7, 7, 7, 7, 103, 7, 101, 7,



2. Page-Fault Frequency (PFF) Scheme

- Keep page-fault rate within a threshold
- If too high $\rightarrow$ allocate more frames or reduce processes
- If too low $\rightarrow$ reclaim frames for other processes

Advantages of Virtual Memory

1. Programs can be larger than RAM
2. Better memory utilization
3. Higher degree of multiprogramming
4. Simplifies programming
5. Efficient use of secondary storage

Disadvantages of Virtual Memory

1. Page fault overhead
2. Thrashing if poorly managed
3. Complex OS and hardware
4. Slower than pure RAM access

Other Issues in Paging / Virtual Memory

1. Prepaging: Prepaging is the process of **loading pages into memory before they are referenced** by the process, to reduce the high number of page faults at startup.

Purpose:

- Avoid the **page fault storm** that occurs when a process starts and its pages are not in memory.

How it works:

- The OS preloads **all or some pages** that the process is likely to need soon.

Advantages:

- Reduces initial page faults → faster process startup.
- Improves overall system performance if prediction is accurate.

Disadvantages:

- Wastes memory and I/O if prepaged pages are **never used**.
- Can reduce frames available for other processes → may cause page faults elsewhere.

Example:

Process requires pages 1,2,3,4,5 in initial instructions:

- Without prepaging → 5 page faults
- With prepaging → pages 1–5 loaded at once → 0–1 page fault

2. Page Size: The **size of a memory page** affects performance, fragmentation, and memory management.

Considerations for choosing page size:**1. Fragmentation:**

- Large pages → more **internal fragmentation** (unused space in page)
- Small pages → less fragmentation, but more pages to manage

2. Page Table Size:

- Smaller pages → more entries → larger page table
- Larger pages → fewer entries → smaller page table

3. I/O Overhead:

- Smaller pages → frequent I/O → higher disk overhead
- Larger pages → fewer I/O operations

4. Locality:

- Page size should ideally match the **process working set**

5. Performance vs Waste Tradeoff

- Balance page size to reduce page faults and minimize unused memory.

3. TLB Reach: The **amount of memory that can be directly mapped through the TLB**.

$$\text{TLB Reach} = \text{TLB Size} \times \text{Page Size}$$

Explanation:

- TLB stores recently used page table entries.

- A larger TLB reach allows **more of a process's working set to reside in TLB**, reducing page table lookups and improving performance.

Ways to improve TLB Reach:

1. **Increase Page Size:** Fewer pages → more memory covered per TLB entry.
 - Tradeoff → increases fragmentation if not all memory is used.
2. **Multiple Page Sizes:** Some processes use large pages, others use small pages → balance TLB coverage and fragmentation.

4. I/O Interlock: I/O interlock ensures that **page frames being used for I/O are not replaced** by the OS until the I/O operation is complete.

Why it's important:

- When a page is **swapped in/out of disk** (e.g., for page fault or prepaging), the I/O may take time.
- If the OS replaces a page **while the I/O is still in progress**, it can **corrupt data** or cause system crashes.

Mechanism:

- OS marks the frame as **“locked”** until the I/O completes.
- During this time, the page **cannot be replaced** by any page replacement algorithm.

Advantages:

- Protects data integrity during disk operations.
- Prevents race conditions and page corruption.

Disadvantages:

- Locked frames reduce the number of frames available → may temporarily increase page faults.
- Excessive I/O interlocks in high-load systems can slightly reduce memory efficiency.

Summary of Issues in Paging / Virtual Memory

Issue	Definition / Purpose	Advantages	Disadvantages
Prepaging	Preload pages before use to reduce startup page faults	Reduces page faults at process start; improves performance	Wastes memory/I/O if unused; reduces frames for other processes
Page Size	Size of a single memory page	Larger pages → smaller page tables, fewer I/O; smaller pages → less fragmentation	Large pages → internal fragmentation; small pages → bigger tables, more I/O
TLB Reach	Memory that TLB can directly map	Improves TLB hit rate; reduces page table accesses	Limited by TLB size; large pages may increase fragmentation
I/O Interlock	Lock frames during I/O to prevent replacement	Ensures data integrity; prevents page corruption	Locked frames reduce available memory; may increase page faults temporarily

Allocating Kernel Memory: The kernel is the core part of the operating system that manages resources, hardware, and processes. Allocating memory in the kernel is different from user programs, because of the Key Characteristics of Kernel Memory

1. Limited Memory:

- The kernel typically has access to **about 1 GB of virtual and physical memory** (on 32-bit systems).

2. Non-pageable:

- Kernel memory **cannot be swapped to disk**. It must remain in **physical RAM** at all times.

3. Physically Contiguous:

- Some kernel operations, especially **Direct Memory Access (DMA)** for devices, require **contiguous physical memory blocks**.

4. Cannot Sleep (Sometimes):

- In some contexts, the kernel **cannot wait or block** for memory allocation. For example, during interrupts or critical sections.

5. High Stakes:

- Mistakes in kernel memory allocation can **crash the system**, unlike user-space programs where a crash only affects one process.

The **main function** used to allocate memory in the Linux kernel is: `kmalloc()`

```
#include <linux/slab.h>

void * kmalloc(size_t size, int flags);
```

- `size` → Number of bytes to allocate.
- `flags` → Control **how the allocation is done**

Example code

```
struct falcon *p;

/* Allocate memory for a structure in kernel */
p = kmalloc(sizeof(struct falcon), GFP_KERNEL);

if (!p) {

    /* Allocation failed, handle error */

    printk(KERN_ERR "Memory allocation failed for falcon struct\n");

}
```

- `GFP_KERNEL` → a **flag** specifying allocation behavior (explained next).
- `p` points to memory in **kernel space** if successful.

Understanding flags

The `flags` parameter tells the kernel how to allocate memory, from where, and under what conditions. It essentially instructs the kernel on:

1. Whether the allocation can block or must be immediate
2. Which memory region (zone) the memory should come from
3. Which combination of the above rules to apply conveniently (type flags)

So, it controls action, zone, and type.

1. Action Modifiers

- Whether the allocation can sleep (block) or not:
 - `GFP_KERNEL` → normal allocation, **can sleep**. Safe in most contexts.
 - `GFP_ATOMIC` → **cannot sleep**, used in interrupt context or other critical sections.

2. Zone Modifiers

- Specify **from which memory region** to allocate:
 - `GFP_DMA` → memory that can be accessed by hardware via DMA.
 - `GFP_HIGHMEM` → memory above low 1 GB (for high-memory systems).

3. Type Flags

- Combine **action** + **zone modifiers** for convenience.
- Examples:
 - `GFP_KERNEL` → typical kernel allocation (may sleep, normal zone)
 - `GFP_ATOMIC` → allocation for critical or atomic context (cannot sleep)

Memory Management with OS Examples

1. Linux Memory Management

- Linux uses a **virtual memory system** that combines **paging**, **demand paging**, and **swapping**.
- Physical memory management is done via the **buddy system allocator**.
- Memory is divided into **zones**:
 - **ZONE_DMA**: For devices needing DMA access.
 - **ZONE_NORMAL**: Regular memory.
 - **ZONE_HIGHMEM**: Memory above 4GB on 32-bit systems.

Key Components:

- **Buddy Allocator:**

- Manages physical memory by splitting it into blocks of size 2^k pages.
- Efficient for allocating contiguous physical pages.
- **Slab Allocator:**
 - Manages kernel objects and small allocations efficiently.
 - Uses caches of preallocated memory chunks.
- **Virtual Memory:**
 - Implements paging with multi-level page tables.
 - Uses **demand paging**, loading pages on fault.
 - Uses **page cache** to reduce disk I/O.
- **Swapping:**
 - Evicts pages to disk if RAM is scarce.
- **Memory Overcommit:**
 - Linux can allocate more memory than physically available, assuming not all will be used simultaneously.
- **Transparent Huge Pages:**
 - Allows larger page sizes to improve performance.

2. Windows Memory Management

- Windows uses a combination of **paging**, **segmentation**, and **demand paging**.
- The kernel manages physical memory with the **Free List**, **Zeroed List**, **Standby List**, and **Modified List** to track page states.

Key Components:

- **Virtual Address Space:**
 - Each process gets a 4 GB address space (on 32-bit systems), split between user mode and kernel mode.
- **Paging:**

- Pages are 4 KB by default.
- Uses **working sets** to control how many pages a process keeps in physical memory.
- **Memory Manager:**
 - Manages page tables, page faults, and swapping.
 - Uses **Page Frame Database (PFN DB)** to track physical pages.
- **Segmentation:**
 - Used mainly for backward compatibility and protection.
- **Heap and Stack Management:**
 - Supports dynamic memory allocation with heaps.
- **Kernel Memory:**
 - Uses special non-pageable pools for critical kernel data.

3. macOS Memory Management

- Built on XNU kernel combining Mach microkernel and BSD.
- Uses **virtual memory with paging** and **wired memory** concepts.

Key Components:

- **Mach VM System:**
 - Virtual memory objects abstract memory regions.
- **Physical Memory Allocation:**
 - Uses a **buddy allocator** for physical memory.
- **Paging and Demand Paging:**
 - Handles page faults by loading pages from disk on demand.
- **Wired Memory:**
 - Memory locked in RAM, cannot be paged out (used for kernel and drivers).
- **Copy-on-Write:**

- For efficient process creation (fork).
- **Memory Zones:**
 - For kernel memory allocation.
- **Speculative Reads:**
 - Prefetches pages expected to be needed.

4. Solaris Memory Management

- Uses **segmentation and paging combined**, often called **segmentation with paging**.
- Physical memory managed through a **page cache**, **anonymous pages**, and **segment tables**.

Key Components:

- **Segment Table:**
 - Maps segments to page tables.
- **Page Cache:**
 - Caches file data in memory to reduce disk I/O.
- **Physical Memory Allocation:**
 - Uses a **page freelist** and thresholds like **lotsfree** and **desfree**.
- **Pageout Daemon:**
 - Frees memory when free pages fall below threshold.
- **Memory Protection:**
 - Uses segmentation to enforce access permissions.
- **Dynamic Tuning:**
 - System adjusts parameters like free memory thresholds based on workload.

5. FreeBSD Memory Management

- Uses a traditional **BSD virtual memory system** with paging and swapping.
- Physical memory managed via a **buddy system allocator**.

Key Components:

- **VM Object Abstraction:**
 - Abstracts memory resources (file-backed or anonymous).
- **Page Queue Management:**
 - Maintains **active**, **inactive**, and **free** page queues.
- **Page Daemon:**
 - Frees pages when free memory is low.
- **Memory Zones:**
 - For efficient kernel memory allocation.
- **Swapping:**
 - Supports swapping out entire processes.
- **Demand Paging:**
 - Loads pages on access faults.

OS	Physical Memory Management	Virtual Memory Model	Key Features	Paging & Segmentation
Linux	Buddy allocator, slab allocator	Demand paging, multi-level PT	Zones (DMA, Normal, HighMem), overcommit	Pure paging (no segmentation)
Windows	Page Frame DB, free/modified lists	Demand paging, segmentation (legacy)	Working sets, page cache, heap management	Paging + legacy segmentation
macOS	Buddy allocator, wired memory	Mach VM system, demand paging	Wired memory, speculative reads	Pure paging
Solaris	Freelist with lotsfree/desfree	Segmentation with paging	Segment table, pageout daemon, tuning	Segmentation + paging
FreeBSD	Buddy allocator, memory zones	Traditional BSD VM, demand paging	VM objects, active/inactive queues	Pure paging

